Undergraduate Project Report
2025/26

From Traditional Operating Systems to AI Agent Operating Systems: Exploring AI-Native OS through the Lens of the MCP Protocol

Name:	Xiheng Li
School:	International School
Class:	2022215107
QMUL Student No.:	221166471
BUPT Student No.:	2022213192
Programme:	Telecommunications Engineering with Management

Date: 05-04-2026

Table of Contents

Abstract 3

Keywords 3

Chapter 1: Introduction 5

1.1 Problem Statement 5

1.2 Our Solution 5

1.3 Main Contributions 6

Chapter 2: Background 8

2.1 Agent Systems and the Limits of User-space Control Planes 8

2.2 What MCP Provides 8

2.3 Limits of User-space Mediation 9

2.4 Relation to Native Linux Security Mechanisms 9

2.5 Related Work 10

2.6 Threat Model 10

Chapter 3: Design and Implementation 13

3.1 Design Goals 13

3.2 System Architecture 14

3.3 Control-Plane Workflow 15

3.4 Wire Protocols 16

3.5 Manifest Semantics and Tool Identity 20

3.6 Session Management and Approval Binding 20

3.7 Kernel Module Design 21

3.8 Daemon Implementation 23

3.9 Tool-Side RPC and Execution Model 23

3.10 State Reconciliation and Recovery 24

3.11 Rationale for the Kernel--User-Space Split 24

3.12 Limitations and Identified Optimisation Opportunities 24

Chapter 4: Results and Discussion 25

4.1 Latency and Control-Plane Overhead 25

4.2 Throughput and Scalability 26

4.3 Security Evaluation 27

4.4 Daemon Failure and Recovery 30

4.5 Discussion 31

4.6 Threats to Validity 32

Chapter 5: Conclusion and Further Work **34**

5.1 Conclusion **34**

5.2 Reflection **34**

5.3 Further Work **35**

References **37**

Acknowledgment **41**

Appendices **42**

Disclaimer **42**

Acknowledgement of using Generative AI **43**

Project specification **44**

Early-term progress report **51**

Mid-term progress report **58**

Supervision log **65**

Additional Appendices **68**

A.1 Wire Protocol Reference 68

A.2 System Comparison Table 71

Risk and Environmental Impact Assessment **72**

Abstract

AI agents increasingly rely on external tools to extend their capabilities, yet existing frameworks lack secure OS-level support for managing tool invocation. When enforcement remains entirely in a user-space gateway, three properties are difficult to guarantee: semantic consistency between a tool's declared and actual behaviour, trusted identity binding resilient to session confusion, and approval-state persistence across gateway failures.

To address this, we present `linux-mcp`, a kernel-assisted control plane for tool invocation on Linux. `linux-mcp` keeps semantic parsing and tool execution in user space while delegating admission control, approval-ticket lifecycle, and binding verification to a Linux kernel module. This partitioning makes approval state durable across daemon failure and inaccessible to unprivileged processes; tool authors need only specify functionality and parameters in a JSON manifest, with no kernel-facing implementation required.

We evaluate `linux-mcp` against a pure user-space baseline and a hardened user-space design across latency, throughput, attack resistance, and failure scenarios. The kernel-assisted path blocks all tested control-plane attacks (110 attempts across four categories) while both user-space baselines permit meaningful bypass rates. This stronger enforcement comes at modest cost: latency overhead is statistically indistinguishable from the baseline at small and medium payloads and is approximately 3.4% *lower* at 1 MB payloads; throughput remains within approximately 10% of the user-space baseline under tested multi-agent workloads. The prototype, comprising 43 tools across 14 applications, is publicly available at `https://github.com/xiheng724/linux_mcp`.

Keywords

AI Agents, Operating Systems, Tool Invocation, System Security

摘要

随着 AI agents 日益依赖外部工具来扩展其能力，现有系统在工具调用管理方面仍缺乏高效且安全的操作系统级支持。对大语言模型（LLM）和外部工具的非受控访问会带来资源滥用、安全漏洞以及执行不可靠等风险，这凸显了对 AI 原生操作系统支持的迫切需求。

为此，我们提出了 `linux-mcp`，一种面向 Linux 操作系统的、内核辅助的工具调用控制平面。该系统引入了一种新的操作系统架构：将语义解析和工具执行保留在用户态，同时将准入控制下沉至 Linux 内核模块中。该设计通过统一的守护进程对所有外部工具调用进行中介，将批准状态持久化于内核中，从而在守护进程崩溃后仍可保持控制平面状态，并与非特权进程严格隔离。基于 MCP 协议的抽象框架支持无缝的工具集成：开发者只需在 manifest 中以声明方式描述工具的功能和参数，无需编写任何面向内核的代码。

我们从延迟、吞吐量、抗攻击能力以及故障场景等方面，将 `linux-mcp` 与纯用户态基线系统和强化用户态系统进行了对比评估。实验结果表明，基于内核的设计能够阻断全部已测试的控制平面攻击，而两种用户态基线均存在明显的绕过率。这一更强的执行保障代价适中：在小负载和中等负载下，延迟与用户态基线无统计显著差异；在 1 MB 负载下反而约低 3.4%；吞吐量在多 agent 并发场景下与用户态基线相差约 10%。该原型系统已在 14 个应用中暴露了 43 个工具，并已开源发布：`https://github.com/xiheng724/linux_mcp`。

关键词

AI 智能体，操作系统，工具调用，系统安全

Chapter 1: Introduction

Recent advances in Large Language Models (LLMs) have enabled instruction following, reasoning, and interaction with external environments, forming the foundation for autonomous agentic systems [1–3]. LLM-based agents are now deployed across a wide range of applications, from direct-acting assistants to sophisticated multi-step reasoning and problem-solving systems [4–7].

1.1 Problem Statement

Tool-using, stateful AI agents expose a fundamental mismatch between modern agent frameworks and the abstractions provided by conventional operating systems [6, 8]. While conventional operating systems effectively manage processes, files, and network resources, MCP-style workflows are organized around higher-level concepts such as semantic tool descriptions, session context, approval boundaries, and cross-step state. This mismatch leads to fragile control planes when enforcement remains entirely in user space. Frameworks such as OpenClaw [9] demonstrate that granting agents execution capability and persistent context can lead to security risks including prompt injection, privilege escalation, and cross-step exploitation.

A single gateway process is typically responsible for interpreting tool manifests, binding requests to identities, and authorizing execution, effectively placing the control plane in user space. Such a design introduces a single point of failure and expands the trusted computing base, making it vulnerable to crashes, logic bugs, or compromised control paths [10]. This makes three properties hard to enforce: semantic consistency between declared and actual tool behaviour, trusted identity binding resilient to replay and session confusion [11], and approval-state persistence across gateway failures [5].

This analysis motivates a central research question: which control-plane invariants in MCP-style tool invocation benefit from kernel-level enforcement, and can such enforcement improve security and resilience without sacrificing performance?

1.2 Our Solution

`linux-mcp` addresses this by moving admission control, approval state, binding checks, and durable observability into a Linux kernel module, while leaving semantic parsing and tool execution in user space. This is not a full AI-native operating system (AI-native OS); the kernel role is deliberately narrow, enforcing only the invariants user space cannot reliably hold.

The architecture has four layers: an LLM client (`llm-app`), a manifest-aware gateway (`mcpd`), a Generic-Netlink-based kernel control plane (`kernel_mcp`), and resident tool services over Unix domain sockets. Every tool invocation follows a fixed path through all four layers; no

request reaches a tool service without first obtaining an explicit kernel admission decision. The component responsibilities are detailed in §3.2, and the rationale for the kernel–user-space split is developed in §3.11.

This design does not reduce the trusted footprint of `mcpd`: the gateway remains fully trusted and is responsible for populating the kernel registries. What it provides instead is a *partitioning* of responsibility: registry entries, approval tickets, and binding checks reside in the kernel, protected by `CAP_NET_ADMIN`-gated Generic Netlink access control, and remain durable across daemon failures and inaccessible to unprivileged processes. Gateway logic bugs therefore no longer threaten the critical state transitions; the kernel enforces them independently of the daemon's correctness. Semantic parsing and tool execution remain in user space, where flexibility matters most.

Rather than replacing existing Linux security mechanisms, `linux-mcp` proposes a kernel-assisted control plane that complements user-space flexibility. AIOS-style proposals offer strong guarantees at the cost of replacing OS infrastructure; user-space frameworks offer broad deployment at the cost of the structural weaknesses identified in Chapter 1.3. The position here is narrower: move only the invariants that require protection from the gateway itself into the kernel, and leave everything else in user space. The central hypothesis is that this targeted placement improves binding safety and failure resilience without imposing prohibitive performance costs; Chapter 3.12 tests this claim empirically.

Comparison with user-space systems. To contextualise the design, we compare `linux-mcp` with two representative user-space tool-invocation frameworks: OpenClaw [9], which uses a monolithic user-space gateway, and OpenHarness [12], which integrates control logic into the agent process itself. Table 1 highlights the key architectural differences.

1.3 Main Contributions

This paper makes the following contributions:

- We design and implement `kernel_mcp`, a Linux kernel module that maintains tool and agent registries, enforces per-invocation admit/deny/defer decisions, and manages approval-ticket lifecycle under `CAP_NET_ADMIN`-gated Generic Netlink access. This makes control-plane state durable across daemon failure and unreachable by unprivileged processes, without placing any JSON parsing or tool execution in the kernel.
- We design and implement `mcpd`, a manifest-aware user-space gateway that loads tool manifests, derives semantic hash identifiers, reconciles tool and agent state with the kernel registry, binds sessions to OS-level UDS peer credentials, validates request payloads against JSON schemas, and forwards tool invocations through the kernel mediation path. `mcpd`

Feature	**linux-mcp**	**OpenClaw**	**OpenHarness**
Control-plane placement	Kernel-assisted	User-space gateway	In-process (same PID as agent)
Trust boundary	Split (user + kernel)	Monolithic (user-space)	Python process boundary
Approval enforcement	Ticket-based (kernel)	User-space logic	In-memory checker
Binding correctness	Kernel-verified (UDS peer creds)	Gateway-managed	No OS-level binding
Failure resilience	State persists in kernel	Lost on gateway failure	Lost on process restart
Security model			
linux-mcp: TCB partitioned; kernel holds durable control-plane state, inaccessible to unprivileged processes.			
OpenClaw: Monolithic TCB; all control-plane state in user-space process.			
OpenHarness: No OS-level enforcement boundary; sensitive-path deny lists enforced only within the Python interpreter.			

Table 1: Key design differences between `linux-mcp`, OpenClaw, and OpenHarness.

is the sole component that bridges manifest semantics and runtime enforcement, keeping this complexity out of both the kernel and the LLM client.

- We conduct a controlled functional evaluation against pure and hardened user-space baselines across latency, throughput, attack resistance, and failure scenarios. The evaluation demonstrates that `linux-mcp` enforces the specified control-plane invariants against a targeted attack set ($n = 30$ runs), quantifies the latency cost of kernel placement, and shows that kernel-held approval state survives daemon failure in a way neither user-space baseline can match.

Chapter 2: Background

2.1 Agent Systems and the Limits of User-space Control Planes

Modern operating systems are built around the process as the primary unit of protection: address-space isolation, controlled file access, and well-defined IPC [13]. These abstractions remain fundamental, but agent-based workloads introduce a layer of intent between the user and execution that they do not directly address. An agent plans over semantically described tools, constructs structured requests, and acts on behalf of a user; correctness now depends not only on process isolation but on whether the semantics of a requested operation match what is actually executed.

Several lines of work have responded to this gap. Systems such as generative agents demonstrate long-lived entities capable of maintaining memory and coordinating workflows [4]. Frameworks including LangChain [14], LangGraph [15], AutoGPT [16], CrewAI [17], and Semantic Kernel [18] structure tool composition in multi-step pipelines. More ambitious proposals embed LLM-centric services directly into a system-level kernel managing scheduling, memory, and tool access [6–8, 19]. At the application layer, `OpenClaw` [9] acts as a persistent user-space control layer for tool execution; Harness [20] embeds governed AI agents in CI/CD pipelines; Apple Intelligence [21] integrates agents with on-device OS services and a private-cloud compute layer.

Section 2.3 details why this enforcement placement matters.

2.2 What MCP Provides

The Model Context Protocol (MCP), released by Anthropic in November 2024, provides a useful abstraction for structuring tool-mediated agent interaction [22]. It defines a client-host-server architecture in which tools, resources, and prompts are exposed through a standardised JSON-RPC interface. The host coordinates sessions and mediates access, while servers implement capabilities; tools are the primary mechanism by which agents take executable actions on external systems. MCP rapidly achieved broad ecosystem adoption, with implementations in Python, TypeScript, C#, and Java, and integration across major LLM providers and agent frameworks [22].

This separation simplifies interoperability and modularity. However, it also concentrates responsibility in the host: identity management, approval decisions, and consistency between tool descriptions and execution all depend on correct host behaviour. MCP defines how components communicate, but does not specify how a local system should enforce these properties. `linux-mcp` implements the enforcement layer that MCP leaves unspecified: it provides the local mechanism that turns the protocol's abstract trust model into a set of kernel-verified invariants.

2.3 Limits of User-space Mediation

Chapter introduced three control-plane invariants; this section explains their technical origin at the MCP protocol layer.

Semantic drift. Tool invocation depends on semantically rich manifests combining natural-language descriptions, schemas, and examples [3, 22, 23]. Because both interpretation and enforcement reside in a mutable user-space process, a change in implementation or a weakened control path may result in executing a capability that no longer matches the approved semantics [24].

Identity anchoring. Agent systems bind requests to sessions or agents rather than PIDs [1, 4, 16], but MCP provides no mechanism to anchor these identities in OS-visible credentials. Prompt injection and session confusion attacks can therefore cause agents to exceed their intended authority even when the gateway appears to enforce correct behaviour [11, 25, 26].

Durability. Approval decisions and pending requests may be lost on crash or restart when control-plane state is held only in memory [8]. The fragility is particularly acute in long-running agent systems where execution spans multiple steps and decisions [27, 28].

These limitations are not unique to any single framework; they reflect a broader pattern in which control-plane invariants [29] are difficult to enforce when confined to user space [30–33].

2.4 Relation to Native Linux Security Mechanisms

Any such design must coexist with existing Linux security mechanisms. Facilities such as seccomp, AppArmor, SELinux, and Landlock provide strong guarantees over system calls, resources, and process privileges [34–38]. These mechanisms operate at the level of kernel objects and access rights.

However, none of them capture higher-level concepts such as tool identity, semantic equivalence between a manifest and an execution endpoint, or the scope of an approval decision. The root cause is a layer mismatch: existing mechanisms enforce policy at the level of kernel objects (files, processes, syscalls), whereas tool identity is an application-layer concept defined by a semantic hash of a JSON manifest. Extending these mechanisms to enforce tool-identity would require either encoding tool hashes into LSM policy (conflicting with the goal of protocol-layer independence) or hooking into the application-protocol layer, which breaks the principle that LSM hooks attach to kernel operations, not to application messages.

This gap motivates introducing a dedicated control layer that validates semantic and identity invariants at the point of tool execution, outside the mutable execution path of the user-space gateway.

2.5 Related Work

Several existing mechanisms address related but distinct problems; understanding why they fall short for MCP-style enforcement motivates the present design.

Existing kernel-native mechanisms such as seccomp-BPF, eBPF/LSM hooks, Landlock, and capability-based designs (Capsicum) provide strong control over syscalls, kernel objects, and process-scoped authority [31, 34, 35, 38, 39]. However, they operate below the application-protocol layer and therefore do not natively express MCP-specific concepts such as manifest-derived tool identity, approval-ticket scope, or semantic equivalence between a declared tool and an executed endpoint. A tool's identity in this system is a SHA-256 digest of its JSON manifest, a concept that does not exist at the syscall or VFS layer; no hook in seccomp or LSM fires at the point where a JSON-RPC payload is interpreted, and the BPF verifier's loop and memory restrictions make runtime hash-registry lookups impractical (see Section 3.11). For this reason, these mechanisms can harden the surrounding execution environment, but they do not directly enforce the control-plane invariants targeted in this work.

Agent frameworks such as LangChain [14], LangGraph [15], AutoGPT [16], CrewAI [17], Haystack [40], and Semantic Kernel [18] locate the full control plane in user space; correctness depends entirely on the in-process runtime remaining uncompromised. `OpenClaw` [9] operationalises this pattern at scale, and independent security analyses confirm the resulting vulnerabilities [25, 26]. Industrial agents such as Harness [20] add governance through platform RBAC, but the trust anchor remains the cloud control plane. Recent work on application-layer mandatory access control —including privilege-escalation frameworks for LLM agents [32] and programmable privilege control for tool-calling agents [33] —enforces policy inside a modified agent runtime, so invariant correctness depends on the runtime itself remaining intact. All of these approaches keep the trust anchor in user space or in a cloud-hosted control plane; their correctness degrades when the runtime or gateway is compromised.

AIOS-style proposals such as AIOS, OS-Copilot, MemGPT, and AgentOS [6–8, 19] pursue a broader scope: they embed LLM-specific abstractions —scheduling, memory management, tool access —into a new OS substrate. These proposals broaden scope by redesigning OS infrastructure; `linux-mcp` instead adds a narrow kernel role to an otherwise unchanged Linux system, targeting a bounded set of MCP-style control-plane invariants.

2.6 Threat Model

The above limitations suggest that the correctness of MCP-style systems depends on a small set of control-plane invariants, including identity binding, approval validity, and consistency between tool semantics and execution. We therefore ground our design in a threat model that focuses on attacks against these invariants in a local-host setting.

Attack class	Representative attack	Defense in `linux-mcp`
Spoofing	Forged session identity or mismatched tool hash	Kernel-verified identity binding and semantic-hash consistency
Replay	Reuse of stale approval tickets or prior requests	Kernel-enforced approval lifecycle with binding and lifetime validation
Substitution	Executing a tool diverging from declared semantics	Verification of manifest-derived semantic identity
Escalation	Bypassing gateway checks via logic flaws	Mandatory kernel arbitration for all invocations

Table 2: Control-plane threats and defences (in scope).

The local-adversary model is appropriate for MCP-style deployments for three reasons. First, tool services in a typical deployment run on the same host as the gateway; the attack surface that most immediately needs hardening is therefore local, not remote. Second, a remote adversary who has already achieved code execution on the host is, by definition, a local adversary: the local threat model subsumes this class without requiring a separate treatment. Third, the two dominant remote-oriented threats in AI agent systems (prompt injection and supply-chain compromise of manifests) operate at different layers and fall outside the scope of kernel-level admission control, as discussed later in this section. The threat model therefore focuses on what kernel enforcement can actually address: a co-located process that can observe, forge, or replay control-plane messages.

Concretely, the adversary model has the following capabilities: (1) arbitrary code execution on the host as an unprivileged process; (2) access to Unix domain sockets exposed by the gateway; (3) ability to observe, craft, and replay JSON-RPC messages; and (4) full knowledge of the MCP protocol, tool manifests, and `linux-mcp` implementation (consistent with Kerckhoffs's principle). The attacker interacts exclusively through the `mcpd` JSON-RPC surface and holds no elevated capabilities: specifically, no `CAP_NET_ADMIN` (required to invoke the kernel module via Netlink), no `ptrace` access to `mcpd`, and no write access to manifest files. These three boundaries are critical: `CAP_NET_ADMIN` is the interface to kernel enforcement; `ptrace` would allow process manipulation; manifest write would break the trusted-input assumption.

Two dominant attack categories in AI agent systems fall outside this scope. *Prompt injection* operates at the model's reasoning layer, before any `tool:exec` request is submitted; kernel admission control has no visibility into the planning process [11, 24]. *Supply-chain compromise of manifests* requires file-system-level defences such as IMA or dm-verity; `linux-mcp` treats on-disk manifests as trusted input and assumes the adversary lacks write access to `tool-app/manifests/`. Also out of scope: process-level attacks on the gateway (ptrace, memory corruption, LD_PRELOAD), kernel-level attacks (rootkits, hardware side-channels), and remote adversaries.

Within this scope, we focus on four classes of control-plane failures: (1) *spoofing* of identity or

session context, (2) *replay* of previously valid requests or approval decisions, (3) *substitution* of tool identity or semantics, and (4) *escalation* through bypassing admission checks in the gateway. These attack classes and their corresponding defences are summarised in Table 2.

These four attack classes directly inform the design of `linux-mcp` and are revisited in Chapter 3.12 through targeted adversarial experiments.

Chapter 3: Design and Implementation

This chapter describes the design and implementation of `linux-mcp`, a kernel-assisted control plane for LLM tool invocation on Linux. The central question driving the design is not how to make the gateway more robust, but which state it should not be solely responsible for holding: `mcpd` retains everything requiring semantic flexibility, while `kernel_mcp` owns the state that must survive gateway failure and remain unreachable by unprivileged processes. The three design goals, component responsibilities, and wire protocols are described in the following sections.

3.1 Design Goals

The design is oriented around three concerns that arise from the problem of mediating LLM tool invocations in a trustworthy manner.

The foremost requirement is that admission control must not reduce to a user-space daemon unilaterally deciding whether to forward a request. The admissibility of a tool invocation is determined solely by explicit kernel-resident state: the registration status of the requesting agent, the presence of the target tool in the registry, and the consistency of its semantic identity with the registered manifest. Anchoring this decision in the kernel ensures it cannot be silently circumvented by tampering with the daemon or by injecting a forged request that the daemon would otherwise accept [39, 41].

At the same time, the system must remain compatible with ordinary user-space tools. Requiring tool authors to write against a kernel-facing API would break compatibility with existing tool ecosystems and make incremental deployment impractical. Tool services therefore carry no kernel-facing obligations; the protocol they implement is described in Section 3.4.

The third concern is observability. Admission, approval, and completion events must not be confined to transient entries in a daemon's in-memory state or application-level logs; they must be queryable through the kernel's own interfaces, independently of whether `mcpd` is currently running. This property is particularly relevant in the context of daemon failure: if `mcpd` crashes, the kernel-held state must remain inspectable without relying on any daemon-side representation.

These goals impose an explicit scope boundary on the design. `linux-mcp` does not attempt to verify tool semantics at runtime, enforce a general policy language, or address every attack surface in LLM-based systems. What it changes is specifically where admission decisions are made and where approval state is maintained.

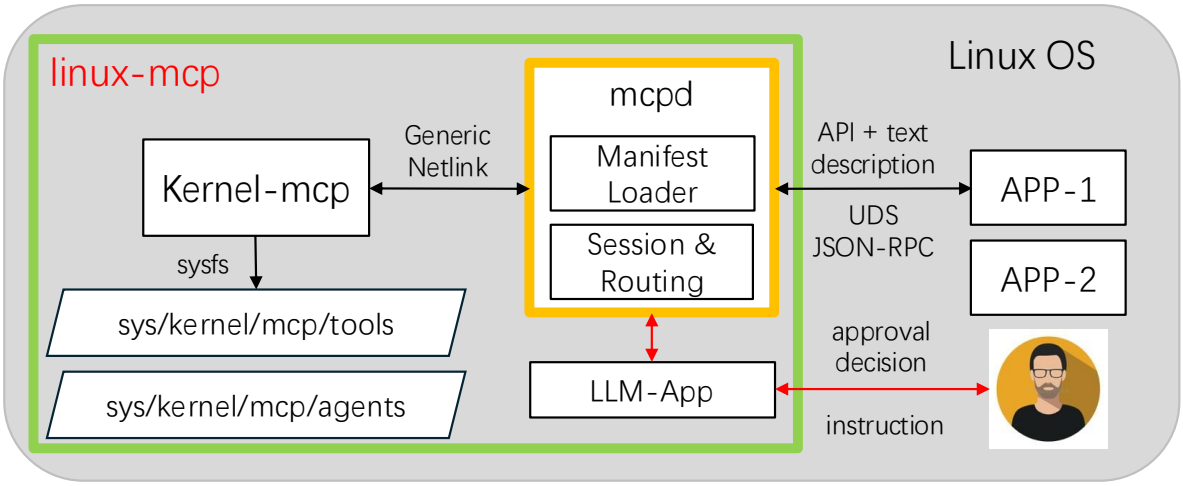


Figure 1: Overall architecture of `linux-mcp`.

3.2 System Architecture

The overall architecture of `linux-mcp`, shown in Figure 1, comprises four components.

`llm-app` is the user-facing client. It queries `mcpd` for the tool catalog, constructs invocation payloads from tool schemas, and submits requests. The client has no direct interaction with the kernel module; all control-plane operations are mediated through `mcpd`.

`mcpd` is the user-space gateway and the semantic front-end of the control plane. It is responsible for loading and validating tool manifests, computing manifest-derived semantic hashes, managing sessions, exposing catalog APIs (`list_apps`, `list_tools`), communicating with the kernel over Generic Netlink, and forwarding admitted requests to tool services over Unix domain sockets (UDS). As the sole component with visibility into both tool semantics and runtime transport endpoints, `mcpd` is correspondingly the only component that requires modification when manifests change or endpoints are reconfigured.

`kernel_mcp` is a loadable kernel module implementing the `KERNEL_MCP` Generic Netlink family. It maintains the tool and agent registries, arbitrates invocation requests, manages approval ticket lifecycles, and exports per-tool and per-agent state through `sysfs` under `/sys/kernel/mcp/`. It neither parses manifests nor executes tools nor implements a general policy language; its sole responsibility is enforcing the compact set of invariants required for admission control.

`tool-app` is the execution side: a collection of resident user-space services, each bound to a Unix domain socket. When a request is admitted, `mcpd` forwards the payload to the appropriate tool service, which executes in an ordinary user-space environment and returns its result to `mcpd`, which then reports completion back to the kernel.

To clarify the separation of responsibilities across layers, Table 3 summarises each component

together with its intended scope and explicitly excluded functionality.

Layer	Component	Main responsibility	Intentionally excluded
Client layer	`llm-app` CLI/GUI	Discover tools, plan usage, open sessions, submit requests, handle approvals	Direct tool access, direct kernel communication
Gateway layer	`mcpd`	Load manifests, compute semantic hash, validate payloads, bind sessions, route requests, communicate with kernel	Durable governance state independent of process lifetime
Kernel layer	`kernel_mcp`	Maintain registries, perform admission decisions, manage approval tickets, expose `sysfs` state	JSON parsing, tool execution, LLM-specific logic
Execution layer	Tool services	Execute application logic and return structured results	Global policy enforcement

Table 3: Concrete layers in `linux-mcp`.

3.3 Control-Plane Workflow

The end-to-end request path is:

$$\texttt{llm-app} \rightarrow \texttt{mcpd} \rightarrow \texttt{kernel_mcp} \rightarrow \texttt{mcpd} \rightarrow \texttt{tool-app} \rightarrow \texttt{mcpd} \rightarrow \texttt{llm-app}$$

Figure 2 illustrates this path as a sequence diagram. A request originates in `llm-app`, which first issues a catalog query that `mcpd` satisfies from its manifest-derived in-memory state. The kernel occupies this path not as a passive observer but as the active arbitration point: `mcpd` cannot forward a request to a tool without first obtaining an `ALLOW` decision from the kernel, and it cannot close an invocation without reporting `TOOL_COMPLETE` back through the same channel. This renders admission and completion as explicit, kernel-visible state transitions rather than implicit control-flow decisions internal to the daemon.

The client uses the catalog response to construct a structured invocation payload and submits it as a `tool:exec` call to `mcpd` (see Table 4 for the message format).

Upon receiving the request, `mcpd` performs a series of user-space checks: it resolves the active session, validates the payload against the declared input schema, identifies the target tool, and retrieves the semantic hash associated with that tool from its manifest-derived state. Only upon successful completion of these checks does `mcpd` issue a `TOOL_REQUEST` to the kernel, supplying the agent identifier, tool identifier, and semantic hash as parameters.

The kernel module then performs arbitration. It verifies that the requesting agent appears in the agent registry, that the target tool appears in the tool registry, and that the semantic hash in the

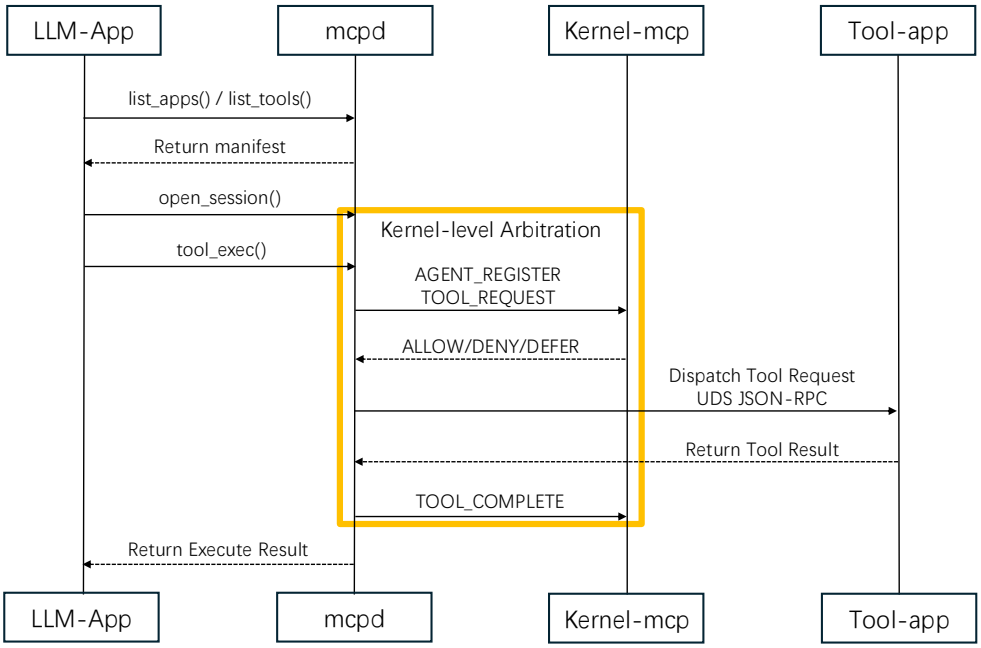


Figure 2: Invocation lifecycle in `linux-mcp`.

request matches the hash recorded at registration time. Based on these checks and the tool's declared policy, the kernel returns a decision that determines whether the request may proceed immediately, must be rejected outright, or requires external approval before execution. The detailed semantics of these decisions are described in Figure 3.

Depending on the kernel's decision, `mcpd` either forwards the request to the tool service, terminates the invocation with an error, or transitions the request into a pending state awaiting explicit approval. After the tool service returns a result, `mcpd` issues `TOOL_COMPLETE` to the kernel, closing the invocation lifecycle from the control plane's perspective.

This workflow has a concrete security consequence: the kernel both opens and closes every invocation. An invocation that never reaches `TOOL_COMPLETE`, whether due to daemon failure or tool timeout, persists as an open entry in kernel state and remains inspectable via `sysfs`. This contrasts with a purely user-space design, in which the lifecycle of a request is only as durable as the daemon's own in-memory representation.

3.4 Wire Protocols

All inter-component communication in `linux-mcp` uses Unix domain sockets with a uniform 4-byte big-endian length-prefixed framing: each message is preceded by a 4-byte header encoding the payload size in bytes, followed immediately by a UTF-8-encoded JSON object. The same framing is used on both the `llm-app`–`mcpd` path and the `mcpd`–`tool-app` path, so a sin-

gle framing library suffices on both sides. The kernel control-plane path uses Generic Netlink (TLV-attribute messages) rather than sockets; that interface is described separately below.

llm-app to mcpd. `llm-app` sends two categories of message to `mcpd`: system control messages (prefixed with a `"sys"` key) and tool execution requests (prefixed with `"kind": "tool:exec"`). Table 4 lists the principal message types and their key fields.

Message	Direction	Key fields
`list_apps`	req	`sys: "list_apps"`
`list_tools`	req	`sys: "list_tools"`, `app_id` (optional)
`open_session`	req	`sys: "open_session"`, `client_name`, `ttl_ms`
`open_session`	resp	`session_id`, `agent_id`, `expires_at_ms`
`tool:exec`	req	`kind`, `req_id`, `session_id`, `agent_id`, `app_id`, `tool_id`, `tool_hash`, `payload`
`tool:exec`	resp	`req_id`, `status`, `result`, `decision`, `reason`, `ticket_id`, `t_ms`
`approval_reply`	req	`sys: "approval_reply"`, `session_id`, `ticket_id`, `decision`, `reason`

Table 4: Principal messages on the `llm-app`–`mcpd` JSON-RPC channel. All messages are framed with a 4-byte big-endian length prefix. The `tool:exec` response carries the kernel's `decision` field (`ALLOW`/`DENY`/`DEFER`) and, on deferral, a `ticket_id` the client uses to issue the subsequent `approval_reply`.

mcpd to tool-app. Once the kernel returns `ALLOW`, `mcpd` forwards the request to the relevant tool service over a separate Unix domain socket connection. The protocol is deliberately minimal: the tool service need not be aware of sessions, agent identities, or kernel tickets. Table 5 lists the five fields in each direction.

mcpd to kernel_mcp (Generic Netlink). The kernel control-plane interface uses the `KERNEL_MCP` Generic Netlink family (version 1). Messages carry TLV-encoded attributes; all registry-mutating commands require `CAP_NET_ADMIN`. The eight commands covering the full invocation lifecycle are listed in Table 6, and the principal TLV attributes carried on the critical `TOOL_REQUEST`/`TOOL_DECISION` path are listed in Table 7.

Field	Dir	Type	Description
req_id	req	*u64*	Per-invocation correlator
agent_id	req	*string*	Requesting agent identifier
tool_id	req	*u32*	Numeric tool identifier from the manifest
operation	req	*string*	Operation name within the tool
payload	req	*object*	Validated input matching the tool's schema
req_id	resp	*u64*	Must echo the request req_id
status	resp	*string*	"ok" or "error"
result	resp	*object*	Tool output on success; {} on failure
error	resp	*string*	Empty on success; error description on failure
t_ms	resp	*u32*	Tool-side execution time in milliseconds

Table 5: mcpd–tool-app JSON-RPC message fields. Tool services implement only this contract; they require no knowledge of sessions, kernel tickets, or the Netlink protocol.

Command	#	Purpose
TOOL_REGISTER	3	Register a tool with its semantic hash and risk flags
AGENT_REGISTER	9	Register an agent with its PID, UID, and binding credentials
TOOL_REQUEST	10	Request admission; kernel replies with TOOL_DECISION
TOOL_DECISION	11	Kernel response: ALLOW (1), DENY (2), DEFER (3)
TOOL_COMPLETE	12	Close the invocation; records execution status and latency
APPROVAL_DECIDE	13	Submit a user approval decision against an open ticket
RESET_TOOLS	14	Clear the kernel-side tool registry (issued on daemon restart)
LIST_TOOLS	8	Dump all registered tools (NLM_F_DUMP mode)

Table 6: KERNEL_MCP Generic Netlink command set. All commands except LIST_TOOLS require CAP_NET_ADMIN.

Attribute	#	Type	Description
`REQ_ID`	1	*u64*	Per-invocation request identifier
`TOOL_ID`	2	*u32*	Numeric tool identifier
`AGENT_ID`	4	*string*	Agent identifier
`TOOL_HASH`	19	*string*	64-char hex SHA-256 of manifest fields
`DECISION`	17	*u32*	Kernel verdict: 1=ALLOW, 2=DENY, 3=DEFER
`TICKET_ID`	22	*u64*	Approval ticket handle (DEFER path)
`APPROVAL_DECISION`	23	*u32*	1=approve, 2=deny, 3=revoke
`AGENT_BINDING`	28	*u64*	Session binding hash (UDS peer credentials)
`AGENT_EPOCH`	29	*u64*	Session epoch; prevents cross-session replay

Table 7: Principal `KERNEL_MCP` TLV attributes. All strings are NUL-terminated and 4-byte aligned. `AGENT_BINDING` and `AGENT_EPOCH` together constitute the session identity embedded in each approval ticket.

Full attribute encodings and a minimal Python framing implementation are in Appendix A.1 (Section 5.3).

Tool registration via manifests. Tool services do not self-register; each application is described by a JSON manifest in `tool-app/manifests/` from which `mcpd` derives semantic hashes and issues `TOOL_REGISTER` Netlink commands at startup. The `endpoint` field is excluded from the hash so a service can be redeployed to a new socket path without invalidating its registration. The complete field specification is in Appendix A.1 (Table 10); a representative manifest is shown below.

Listing 1: Representative tool manifest (JSON). The `endpoint` field is excluded from the semantic hash; all other fields shown here are included.

```
{
  "app_id": "sys_app",  "transport": "uds_rpc",
  "endpoint": "/tmp/linux-mcp-apps/sys_app.sock",
  "tools": [{
    "tool_id": 1,  "name": "get_system_info",
    "risk_tags": ["sensitive_read"],
    "description": "Return OS version, CPU, and memory summary.",
    "input_schema": {"type": "object", "properties": {}},
    "approval_policy": null,  "examples": [{"payload": {}}]
  }]
}
```

3.5 Manifest Semantics and Tool Identity

A control plane that arbitrates based on tool identity requires a stable definition of what constitutes the logical identity of a tool. Tying identity to transport details (socket paths, port numbers, or endpoint URIs) causes routine operational changes to generate spurious re-registration events and demands policy reconciliation on every redeployment. Conversely, if identity is defined too coarsely, the system cannot distinguish a tool that has been semantically modified from one that has simply been redeployed at a different address.

`linux-mcp` resolves this tension by separating semantic identity from transport location. Each tool is described by a JSON manifest containing logical attributes: tool identifier, tool name, application identifier, application name, risk tags, description, input schema, examples, path semantics, and approval policy. These fields describe what the tool is intended to do, not where it runs. `mcpd` canonicalises the semantically relevant field subset into a stable JSON representation and computes the full 256-bit SHA-256 digest over it, represented as a 64-character hexadecimal string. The full digest is used without truncation: a truncated prefix (e.g., 8 hex characters, 32 bits) would allow collisions by the birthday bound after approximately 2^{16} tool registrations, which is not a large number in a production deployment spanning many applications. Crucially, the Unix domain socket endpoint is excluded from the hash computation. A tool may therefore be redeployed to a new socket path without altering its logical identity, provided the semantic fields in the manifest remain unchanged. Conversely, any modification to a semantically significant field (the input schema, risk tags, description, approval policy, or path semantics) produces a different 256-bit digest, which the kernel detects as a mismatch upon the next `TOOL_REQUEST`.

This separation is essential for admission control: the kernel and the daemon can only meaningfully agree on whether a request is legitimate if they share a common, stable notion of tool identity. The semantic hash is the concrete artefact that makes this agreement possible: computed in user space but verified in kernel space, so the daemon cannot substitute a different tool without producing a detectable mismatch.

3.6 Session Management and Approval Binding

While deferring and resolving a request is mechanically straightforward, binding an approval to the exact session, requester, and operation that triggered it requires explicit ticket-level state and precise matching logic to prevent TOCTOU violations.

In `linux-mcp`, each session maintained by `mcpd` is bound to the real peer credentials of its Unix domain socket connection [42]. From these credentials, the daemon derives a *binding hash* and a *binding epoch*. When a request enters the `DEFER` state, the kernel creates an approval ticket and associates it with this binding context. Upon receiving an `APPROVAL_DECIDE` message, the kernel verifies that the binding context stored in the ticket still matches the current session state

before permitting the deferred request to proceed.

This mechanism ensures that approvals cannot be replayed across sessions or misdirected to structurally similar but distinct requests. More importantly, it guarantees that the system state validated at deferral time is identical to the state at execution time, eliminating time-of-check-to-time-of-use (TOCTOU) inconsistencies [43, 44].

Beyond basic ticket binding, the implementation exposes a more nuanced structure: `linux-mcp` employs *two layers of arbitration* and supports *two distinct approval paths*.

(1) Kernel-mediated approval path (DEFER ticket flow). In this path, the kernel acts as the authoritative control-plane arbiter. A `TOOL_REQUEST` may result in one of three decisions:

- `ALLOW`: the request proceeds directly to execution.
- `DENY`: the request is rejected immediately.
- `DEFER`: the kernel suspends execution and issues an approval ticket.

When a request is deferred, `mcpd` records the pending operation and returns the ticket identifier to the client. After the user approves, `mcpd` first sends `APPROVAL_DECIDE` to the kernel to update the ticket state, and only then replays the original request with the associated ticket identifier. The kernel consumes the ticket during the second `TOOL_REQUEST` and permits execution to proceed.

(2) User-space policy path (local approval). Separately, `linux-mcp` supports a local approval mechanism driven by the `approval_policy` field in the manifest. In this case, `llm-app` may prompt the user for confirmation *before* issuing a request to `mcpd`. If the user declines, the request is rejected locally without entering the kernel arbitration path, and no approval ticket is created. This separation allows low-cost semantic checks to be handled in user space while reserving kernel mediation for higher-risk operations.

Figure 3 illustrates the complete state machine across both layers. The key invariant is that approval resolution occurs *before* tool execution: no deferred request can bypass the approval gate, and every execution corresponds to an explicitly consumed ticket. The kernel need not understand the full meaning of a request; it must only verify that the binding context is preserved, the ticket is valid and unconsumed, and the `DEFER` → `ALLOW` transition is explicitly authorised [45].

3.7 Kernel Module Design

To enforce the state transitions and ticket lifecycle described in the preceding sections, `kernel_mcp` is implemented as a loadable kernel module exposing the `KERNEL_MCP` Generic Netlink family. The command set covers the full invocation lifecycle: `TOOL_REGISTER`, `AGENT_REGISTER`, `TOOL_REQUEST`, `TOOL_DECISION`, `TOOL_COMPLETE`, `APPROVAL_DECIDE`, `RESET_TOOLS`, and `LIST_TOOLS`.

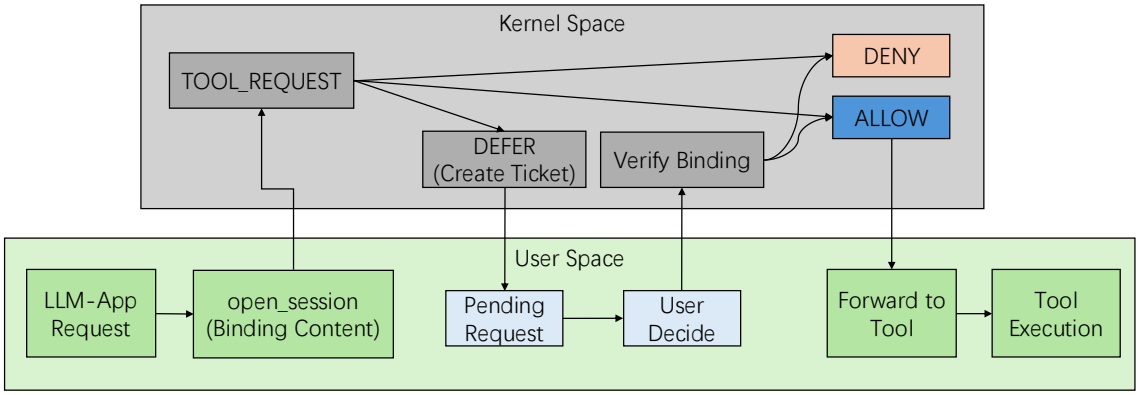


Figure 3: Approval-ticket flow in `linux-mcp`: the `DEFER`–`APPROVE`–`ALLOW` state machine enforced by the kernel module.

Generic Netlink was preferred over alternatives such as `ioctl` or character devices for three reasons: it provides a naturally structured message format, supports dump-style enumeration for registry introspection, and yields a self-describing kernel interface well suited to a research prototype [46].

Internally, the module maintains two registries. The tool registry maps tool identifiers to their manifest-derived semantic hashes and associated metadata. The agent registry records the identifiers of entities permitted to submit requests through the control plane. Approval tickets are heap-allocated kernel objects with explicit lifecycle fields tracking whether a ticket has been decided, approved, consumed, or expired.

The arbitration logic is deliberately narrow. For a `TOOL_REQUEST` to be admitted, three conditions must all hold: the submitting agent must appear in the agent registry, the target tool must appear in the tool registry, and the semantic hash supplied by `mcpd` must match the hash recorded at registration time. If the tool's manifest marks it as requiring explicit approval, the request is deferred rather than immediately allowed.

The module also exports state through `sysfs`. Under `/sys/kernel/mcp/tools/`, each tool has a directory containing its name, hash, and associated metadata. Under `/sys/kernel/mcp/agents/`, per-agent entries record counters such as `allow`, `defer`, `completed_ok`, and `last_exec_ms`, together with a human-readable `last_reason` field. This observability is valuable both during development and during evaluation: the experimenter can verify which tools are registered, confirm which hashes the kernel regards as authoritative, and inspect the kernel's view of agent activity independently of any daemon-side logs.

3.8 Daemon Implementation

While the kernel enforces the critical state transitions, `mcpd` is where the majority of the system's substantive work takes place. Its implementation is organised around four modules: manifest loading and semantic hashing, session storage and binding-context management, Netlink communication with the kernel, and request forwarding to tool services.

The manifest loader is the semantic entry point of the system, producing both the kernel registry entries and the catalog served to `llm-app` in response to `list_apps` and `list_tools` queries.

The session store maintains active connections together with their peer-derived binding information and any pending approval context. This gives the daemon sufficient information to associate incoming approval decisions with the correct deferred request, without requiring the kernel to store full session semantics. The binding information, derived from Unix domain socket peer credentials, is what gets embedded in approval tickets.

The Netlink client provides a persistent communication channel to the kernel module. A dedicated, long-lived socket is maintained rather than opening a new connection per request; this keeps the control path measurable and avoids repeated socket-setup overhead. Per-stage timing covering session lookup, kernel arbitration, tool execution, and end-to-end latency can optionally be recorded for each invocation, which is useful for separating control-plane overhead from tool-side processing time during evaluation.

The server-facing side of `mcpd` handles two distinct roles: it exposes the catalog and invocation interfaces consumed by `llm-app`, and it forwards admitted requests to the appropriate `tool-app` service via the same Unix domain socket transport used on the tool side.

3.9 Tool-Side RPC and Execution Model

The tool-side protocol (detailed in Table 5) is intentionally minimal: each tool service is a resident process bound to a Unix domain socket with no kernel-facing obligations. Because the boundary between admission and execution is drawn at the kernel, the boundary within user space is equally unambiguous: the kernel decides whether a request may proceed; everything concerning what the request means and how it is executed remains in user space. Failure handling follows naturally from this separation. If a tool crashes, times out, or returns an error, `mcpd` handles the event in user space and reflects it back to the kernel through `TOOL_COMPLETE`, marking the invocation lifecycle as closed. The kernel requires no visibility into how the tool failed internally.

3.10 State Reconciliation and Recovery

A design that splits state across the kernel and a user-space daemon must handle the case where the two sides diverge. `mcpd` may restart, intentionally during development or unintentionally following a crash, while the kernel module remains loaded with its existing registry state. If manifests change between a crash and a restart, the tool hashes registered in the kernel will no longer match the hashes derived from the current manifest files, and subsequent `TOOL_REQUEST` messages will be denied with a `hash_mismatch` error.

`linux-mcp` addresses this with a conservative reconciliation strategy. On startup, `mcpd` issues a `RESET_TOOLS` command to clear the kernel-side tool registry, then re-registers the full set of tools from the current manifest state. The system unconditionally rebuilds state from the manifest source of truth rather than attempting to assess whether residual kernel state remains valid. This trade-off is appropriate for a research prototype, where state correctness and auditability take precedence over minimising re-registration overhead.

Session state is lost on crash, since the binding context for in-progress sessions exists only in the daemon process; approval tickets and registry state held in the kernel are not. The durability of that kernel-held state across daemon failure is verified experimentally in Section 4.4.

3.11 Rationale for the Kernel–User-Space Split

The preceding sections establish what the kernel holds and how it enforces it; this section explains why Generic Netlink was chosen as the kernel interface rather than the three most plausible alternatives.

Three kernel-interface designs were considered before settling on Generic Netlink. LSM hooks fire at the syscall boundary and cannot natively correlate a Netlink message with a registered tool hash. eBPF is constrained by the verifier and cannot maintain the ticket lifecycle state the design requires without reintroducing a user-space dependency. A capability model would require tool authors to write against a kernel-facing API, violating the goal that tool services need no modification to participate in the control plane. Generic Netlink supports structured messages, dump-style registry enumeration, and full lifecycle state management without any of these constraints, at the cost of requiring a loadable module rather than a policy hook.

3.12 Limitations and Identified Optimisation Opportunities

The current implementation makes five deliberate simplifications appropriate for a prototype: a global registry lock, unconditional `RESET_TOOLS` reconciliation on restart, per-registration Netlink round-trips, no hot-reload path, and no approval-ticket TTL or garbage collection. Each has a natural resolution direction; these are elaborated in Section 5.3.

Chapter 4: Results and Discussion

`linux-mcp` is evaluated along four dimensions: control-plane latency and its breakdown across invocation stages, throughput under concurrent load, resistance to a representative set of control-plane attacks, and system behaviour under daemon failure and restart. The central claim under test is that relocating admission-control state into the kernel strengthens enforcement without making the tool invocation path prohibitively expensive. Two baselines isolate this effect from user-space hardening alone: a conventional user-space mediator (**userspace**), in which registration, admission, and approval all reside in the daemon, and a hardened variant (**seccomp**) that augments the same control path with stricter syscall filtering, additional validation, and audit logging.

The prototype under test comprises 43 tools across 14 applications; all three systems operate over this same tool deployment throughout the evaluation. All experiments were conducted on a VMware guest (Apple aarch64 host, 4 vCPU, 8 GB RAM) in a headless configuration with no concurrent user-space workloads. Latency results are reported over $n = 30$ independent runs of 2000 requests each, giving a standard error of the mean small enough to support Welch's t-test comparisons at $\alpha = 0.05$ while remaining within the hypervisor timing noise of a virtualised guest; an earlier $n = 5$ pilot is retained as an archived dataset but is not used for numerical claims. Throughput and the category-level security evaluation are drawn from the same run, using the harness's built-in concurrency sweep and ten repeats of each attack case. All reported p-values are two-sided Welch's t-tests [47] comparing per-repetition mean latency against the `userspace` baseline.

4.1 Latency and Control-Plane Overhead

Table 8 and Figure 4 report mean end-to-end latency for each system at three payload sizes. At 100 B, the kernel path and the user-space baseline are statistically indistinguishable: both mean to 0.854 ms ($p = 0.984$). At 10 KB the kernel path is nominally slower (0.899 ms versus 0.863 ms, +4.2%), but the difference does not reach statistical significance ($p = 0.097$), consistent with a sub-millisecond effect lying below the timing noise floor of a VMware guest at $n = 30$.

The 1 MB result is more definitive. The kernel path (7.487 ms) is measurably faster than the plain user-space baseline (7.748 ms, a 3.4% reduction, $p = 0.00027$), and considerably faster than the hardened variant (9.843 ms). The direction of this difference is the opposite of the intuition that any additional privilege boundary must increase latency. The seccomp penalty at large payloads reflects the accumulating cost of BPF per-syscall filtering across the heavier socket traffic induced by megabyte-class payloads; the kernel path does not incur this cost. The modest speedup relative to plain user-space most likely reflects early-reject behaviour: the

arbitration decision is made before the payload is copied through the tool-forwarding socket, so a rejected request avoids the copy that the user-space baseline would otherwise perform between admission and forwarding.

The p_{95} picture mirrors the mean: at 100 B and 10 KB the three systems are statistically indistinguishable ($p = 0.51$ and $p = 0.16$ for the kernel versus userspace comparison); at 1 MB the kernel path records a p_{95} of 8.422 ms versus 8.716 ms for userspace ($p = 0.049$) and 10.903 ms for seccomp, confirming that the tail does not widen under the kernel-assisted path. Figure 8 shows the p_{95} values across all three payload sizes.

Stage-level breakdown in Figure 6 confirms that kernel arbitration is a minor contributor at all payload sizes. Mean arbitration time is 0.028 ms at 100 B and 10 KB, and 0.038 ms at 1 MB. At 1 MB, tool execution accounts for 89.3% of total end-to-end time on the kernel path (89.9% on the user-space path); the entire control-plane machinery (Generic Netlink round-trip, session lookup, and arbitration) accounts for the remaining 10.7%, with the arbitration contribution within that fraction less than four hundredths of a millisecond.

A supplementary microbenchmark (Figure 7) isolates the Generic Netlink layer. Bare round-trip time averages 8.196 μs; the full `TOOL_REQUEST` path, which includes tool and agent registry lookups, averages 9.315 μs. The lookup overhead is approximately 1.1 μs, or 12% of the bare RTT. Kernel-side arbitration therefore costs roughly one order of magnitude less than the control-plane overhead visible in end-to-end measurements, confirming that the dominant cost on the kernel path originates in user-space components (session management, payload serialisation, and socket I/O) rather than in the kernel module itself.

Table 8: Mean end-to-end latency (ms) by payload size ($n = 30$ runs $\times$ 2000 requests). Bold values indicate the lowest latency per row. p-values are two-sided Welch's t-tests against the `userspace` baseline; ^{ns} = not significant ($p > 0.05$).

	Mean latency (ms)			**Welch's p-value**	
Payload	`userspace`	`seccomp`	`kernel`	vs. `seccomp`	vs. `kernel`
100 B	**0.854**	0.867	**0.854**	0.475^{ns}	0.984^{ns}
10 KB	0.863	0.922	0.899	0.000206	0.097^{ns}
1 MB	7.748	9.843	**7.487**	$<10^{-4}$	0.00027

4.2 Throughput and Scalability

Throughput was measured across a range of agent counts (1, 5, 10, 20, 50) and concurrency levels (1, 10, 50, 100) to assess whether the kernel-assisted control plane becomes a bottleneck

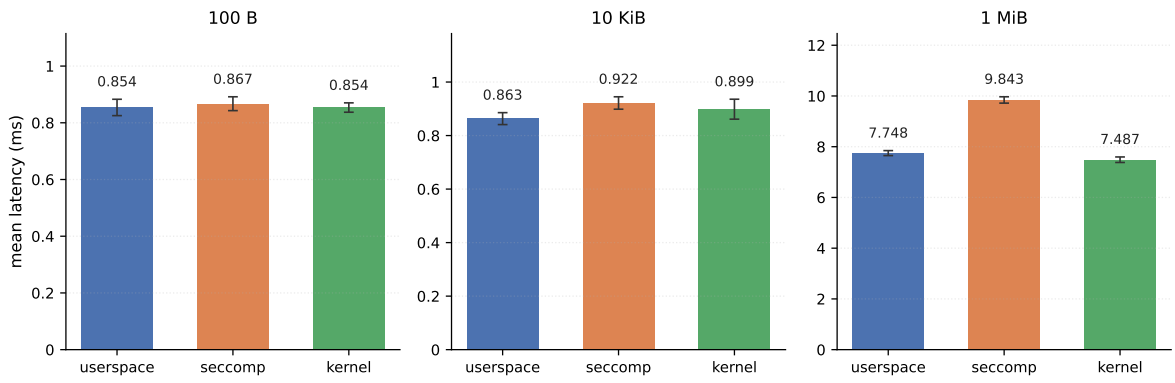


Figure 4: Mean end-to-end latency by payload size for the three systems. Each subplot auto-scales its own y-axis so that the sub-millisecond differences at 100 B and 10 KiB are not flattened by the much larger absolute values at 1 MiB. Error bars show 95% confidence intervals across $n = 30$ independent runs; significance markers are Welch's t-test against the `userspace` baseline.

under load.

The user-space baseline achieves the highest peak throughput, ranging from 1090 to 1219 RPS across configurations. The kernel path ranges from 1003 to 1137 RPS, and the seccomp path from 1020 to 1200 RPS. At a representative operating point (one agent, concurrency 50), userspace achieves 1215.6 RPS, seccomp 1123.5 RPS, and the kernel path 1096.2 RPS. The kernel path is approximately 10% slower than the user-space peak, consistent with the additional Generic Netlink round-trip and the serialisation of admission decisions through the kernel module. (These figures are drawn from an earlier $n = 5$ pilot run; the $n = 30$ study focused exclusively on latency and did not repeat the concurrency sweep. The throughput results should therefore be interpreted as exploratory evidence of practical viability rather than a definitive performance characterisation: they establish that the kernel-assisted path does not collapse under concurrency, but the specific figures carry wider uncertainty than the latency results.)

The figures report mean throughput only; tail-latency evolution under sustained load (p_{99}, p_{95}) would require a separate concurrency sweep and is not characterised here. Within the tested range, kernel arbitration adds approximately 10% overhead without producing a throughput collapse, establishing practical viability rather than a definitive performance ceiling.

4.3 Security Evaluation

Performance is secondary to the question of whether kernel placement actually strengthens admission control. The attack evaluation targets four categories corresponding to the threat model introduced in Chapter 1.3: identity spoofing (forging or borrowing session identity to impersonate a registered agent), replay (reusing a stale approval ticket across a new session or after expiry), substitution (tampering with the `tool_id` or `tool_hash` in a live request to redirect

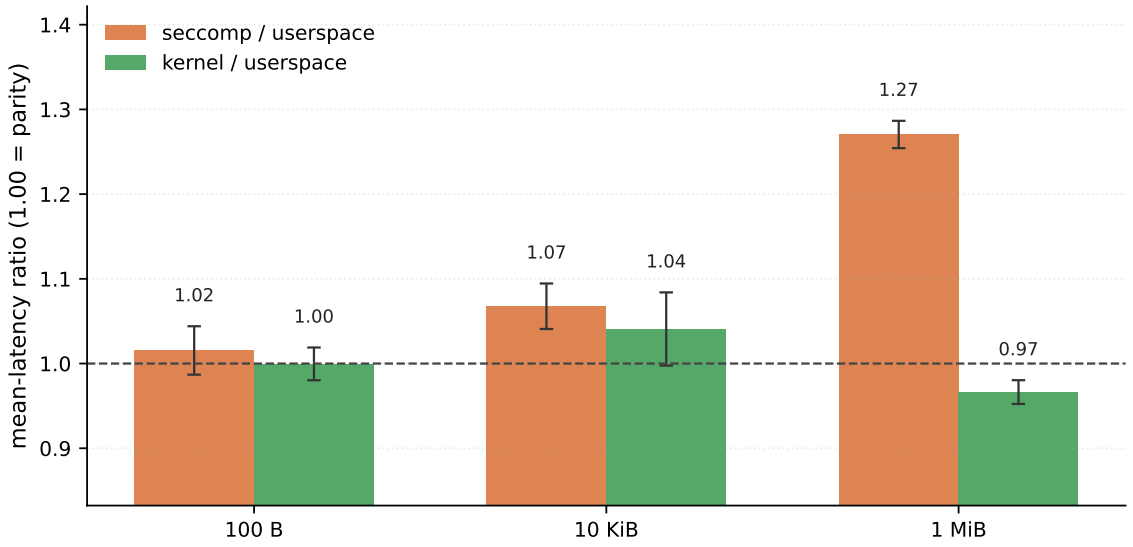


Figure 5: Mean-latency ratio of each hardened variant to the plain `userspace` baseline, with 95% confidence intervals propagated from the 30-run per-repetition means. The y-axis is zoomed around the parity line (ratio = 1.00) to make the sub-4% effects at 100 B and 10 KiB visible alongside the larger effect at 1 MiB. The `seccomp` path is significantly slower at every payload size; the `kernel` path is statistically indistinguishable from `userspace` at small and medium payloads and measurably faster at 1 MiB.

execution), and escalation (bypassing the approval gate so that a high-risk invocation proceeds without explicit confirmation).

Each system received 110 attack attempts in total (11 distinct cases × 10 repeats: 3 spoof, 4 replay, 3 substitution, 1 escalation). Outcomes are summarised in Table 9 and Figure 9.

Table 9: Category-level attack outcomes by system (110 attempts each: 3 spoof, 4 replay, 3 substitution, 1 escalation, × 10 repeats). The bypass rate measures the fraction of attempts that succeeded in circumventing the intended control. The user-space bypass rates reflect deliberate per-category instrumented weakening of the mediator and do not represent the performance of a fully hardened user-space implementation.

System	Bypasses	Total	Bypass rate
`userspace`	100	110	90.9%
`seccomp`	40	110	36.4%
`kernel`	0	110	0.0%

The plain user-space baseline fails to block 100 of 110 attempts (90.9% bypass rate). This result requires careful interpretation: the harness does not attack an arbitrary user-space mediator, but one whose code paths are deliberately weakened in per-category instrumented modes (`tamper_session`, `tamper_approval`, `tamper_metadata`, `compromised_userspace`) so that

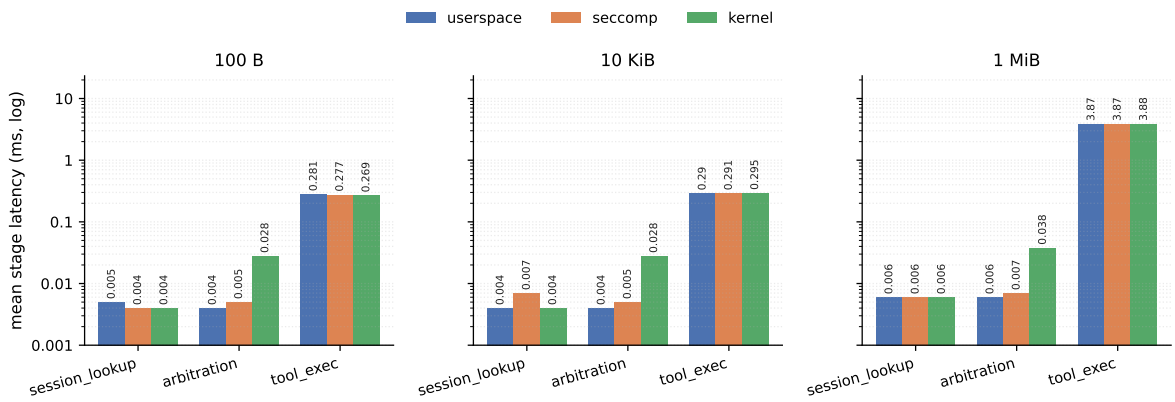


Figure 6: Invocation-stage breakdown (session lookup, arbitration, tool execution) for each system at all three payload sizes, on a log y-axis. The log scale makes session_lookup (around 4 µs) visible alongside tool_exec (around 4 ms). The `kernel` path spends roughly one order of magnitude more time in arbitration than the user-space baselines, but that extra cost is well under 0.04 ms and is invisible relative to the tool-execution component.

each test case exercises the specific check it is designed to evade. The 90.9% figure therefore reflects what a user-space mediator cannot structurally protect against when its own state is under attack, not what a fully hardened user-space system could achieve if those checks were unrelaxed. The hardened variant reduces the bypass rate to 36.4% through stricter checks and a reduced syscall surface, but because admission remains a user-space decision, attacks that need not cross a privilege boundary (particularly substitution and spoofing) still succeed at meaningful rates.

The kernel path blocks all 110 attempts. Spoofing fails because agent identity is tied to a kernel-registered identifier that the control plane verifies independently of the daemon's session state. Replay fails because approval tickets are kernel-resident lifecycle objects with explicit `consumed` and `expired` fields; once a ticket has been advanced or has passed its validity window, no subsequent `APPROVAL_DECIDE` can reopen it. Substitution is blocked by the semantic hash check: regardless of whether an attacker modifies the `tool_hash` field in the request or replaces the tool endpoint at runtime, the kernel compares the submitted hash against the one recorded at registration and rejects any mismatch. A supplementary injection experiment confirms this independently: in 30 live `llm-app` → `mcpd` → `kernel` execution chains, every runtime hash substitution produced a `DENY` with `reason=hash_mismatch` at the kernel layer, regardless of which tool the LLM planner had selected.

One implementation detail of the ticket lifecycle strengthens the replay defence beyond what the category table directly measures: a ticket is bound not only to the session, agent, tool, and hash, but also to the exact `req_id` of the issuing request. A replay consume attempt that reuses a different `req_id` returns `approval_ticket_scope_mismatch`, making it substantially harder to exploit a stolen ticket identifier in a fresh request context.

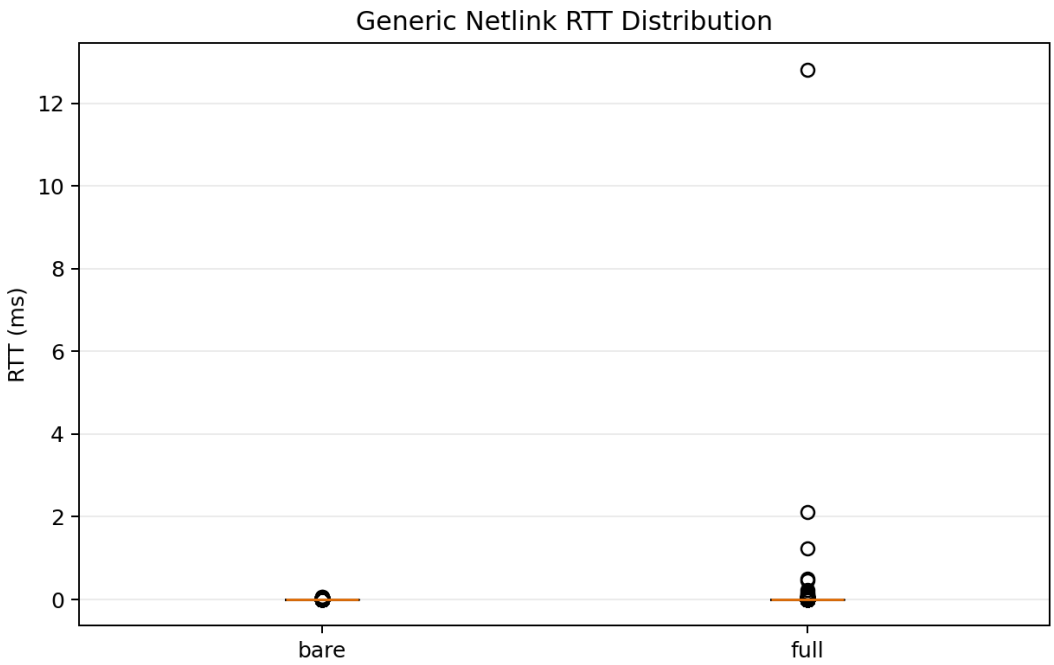


Figure 7: Generic Netlink microbenchmark: bare round-trip time versus the full `TOOL_REQUEST` path. Adding tool and agent registry lookups increases latency by approximately 1.1 μs, keeping the full path within the same microsecond-scale range as the bare RTT.

Three caveats bound the interpretation of Table 9. First, all 110 attack attempts were authored by the same person who built the defences, and each category maps one-to-one onto an enforcement mechanism in the kernel module. A 0% bypass rate on the kernel path is therefore a functional-correctness test of four specified invariants against inputs constructed to exercise exactly those invariants, not evidence of adversarial robustness against unknown attacks.

Second, the combined attack set is scoped to the stated control-plane invariants and does not include: attacks assuming compromise of the kernel module itself; attacks exploiting semantic vulnerabilities in tool logic; attacks requiring already-obtained privileged capabilities (such as `CAP_NET_ADMIN`); or attacks against the LLM prompt-construction pipeline. The 0% bypass rate is a statement about the specified control-plane invariants, not about the full attack surface of an LLM-integrated system.

Third, the user-space bypass rates reflect the instrumented tamper modes described above, not the performance of an optimally hardened user-space implementation.

4.4 Daemon Failure and Recovery

A core architectural argument for kernel-held state is that it should outlive the process that registered it. To test this property, the daemon was forcibly interrupted mid-operation and control-plane state was observed before and after restart.

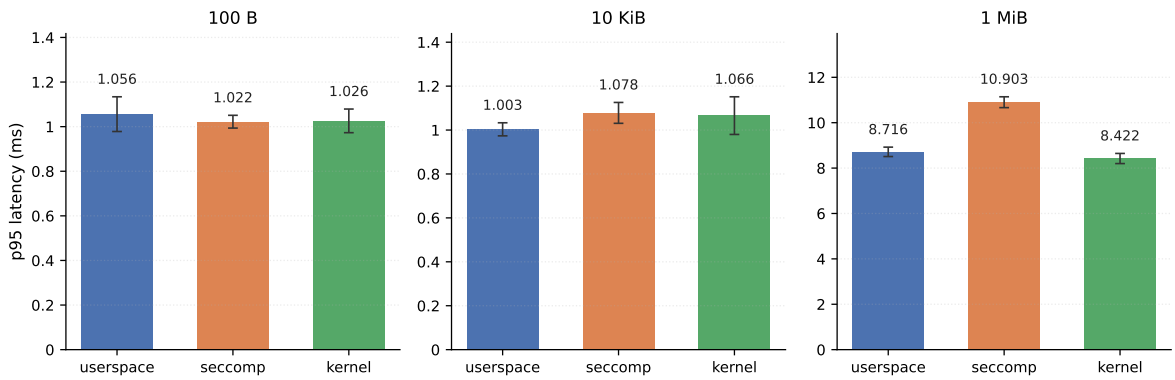


Figure 8: p_{95} latency by payload size for the three systems ($n = 30$ runs of 2000 requests each). Error bars show 95% confidence intervals. At 100 B and 10 KB the kernel path is statistically indistinguishable from the userspace baseline; at 1 MB the kernel p_{95} (8.422 ms) is lower than both userspace (8.716 ms, $p = 0.049$) and seccomp (10.903 ms), confirming that the enforcement boundary does not widen the tail under the tested workload.

In both user-space systems, daemon failure is equivalent to control-plane failure: registered tool state, in-flight session context, and pending approval tickets are all bound to the process that held them. After restart, none of this state is externally accessible; recovery amounts to either starting over or reloading from application-level logs, with no guarantee of consistency.

In the kernel path, the picture is different in a practically important way. Approval tickets, the tool registry, and agent registrations survive the failure intact and remain inspectable via `sysfs` without any daemon involvement. Session state is not preserved, since the binding context for active sessions is owned by the daemon process and is lost with it, but this is expected as session state is inherently tied to an active connection. The durability of deferred approvals and registry state allows `mcpd` to reconcile its manifest-derived view against the kernel-held view on restart and re-establish a consistent catalogue without issuing duplicate registrations.

The reconciliation strategy on restart is conservative (described in Section 3.10): `mcpd` clears and fully rebuilds the tool registry from manifests rather than attempting to reuse residual kernel state. An incremental approach that re-registers only changed tools would reduce restart overhead and is a natural direction for future work.

4.5 Discussion

The three sets of results are more informative read together than separately.

The security and performance results are structurally linked. The seccomp baseline narrows the syscall surface but keeps the admission decision inside a mutable user-space process, which is precisely why it achieves only 36.4% attack resistance despite its overhead: adding BPF filters does not change where trust is anchored. The kernel path achieves 0% bypass not because it filters more aggressively, but because the state that determines admission cannot be reached

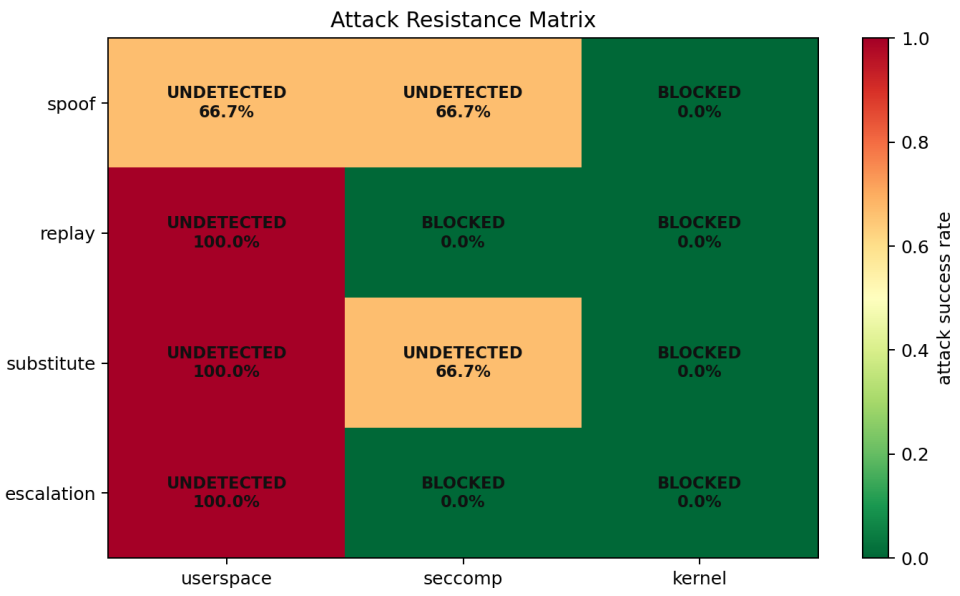


Figure 9: Attack bypass rate by category and system. Each cell shows the absolute count (successes/attempts) and the percentage bypass rate; darker cells indicate a higher bypass rate. The `kernel` column is uniformly zero across all four categories, while the two user-space baselines permit bypasses on the specific checks that their instrumented tamper profiles relax.

by an unprivileged process at all. The same architectural property explains the 1 MB latency inversion: BPF filtering accumulates cost per syscall as payload size grows, while kernel arbitration cost is independent of payload size. The overhead of the stronger enforcement boundary is therefore not additive with workload; it is bounded.

The durability result reveals a property that neither security nor performance benchmarks surface directly. Approval tickets and registry state surviving daemon failure is not an optimisation; it is a consequence of where the state lives. A user-space design cannot achieve this without an external persistence layer, which reintroduces consistency and recovery complexity that the kernel path avoids by design. This matters most in the scenario that deployment experience suggests is most common: a daemon crash or forced restart mid-workflow.

The main open question is not correctness but scalability. The global registry lock is sufficient for the concurrency levels tested, but whether it remains so at 500 or 1000 concurrent requests with many simultaneously deferred approvals is not established. Per-tool or per-agent lock decomposition is the natural next step, but it is future work rather than a gap in the current evaluation.

4.6 Threats to Validity

The results should be interpreted within four scope limitations: all experiments ran in a single VMware guest on an aarch64 host, so latency figures are environment-specific and generali-

sation to bare-metal or different kernel configurations is not established; the evaluation uses demonstration tools with predictable execution times and does not characterise tail latency under sustained load; the attack study is a controlled functional-correctness test of the four explicitly identified invariants, with all attempts authored by the same person who built the defences, and does not constitute a robustness assessment against unknown attacks, prompt injection, or privileged adversaries; and kernel-level admission control addresses only one layer of an AI-agent system, leaving semantic tool vulnerabilities, unsafe policy choices, and compromise outside the stated threat model unaddressed.

Chapter 5: Conclusion and Further Work

5.1 Conclusion

This project set out to identify which parts of an MCP-style control plane genuinely benefit from kernel-level enforcement. The answer is a small and well-defined set: semantic identity, requester binding, approval validity, and invocation completion. These four invariants are structurally difficult to hold when enforcement is confined to a user-space gateway.

`linux-mcp` moves only those invariants into a Linux kernel module, while leaving semantic parsing and tool execution in user space. The gateway remains fully trusted, but the state it establishes now resides in the kernel, beyond the reach of unprivileged processes and durable across daemon failure.

In the tested prototype setting, this partitioning blocked the full set of 110 targeted control-plane attack cases in our functional evaluation, where both user-space baselines permitted meaningful bypass rates. Latency remained statistically indistinguishable from the user-space baseline at small and medium payloads, was measurably lower at 1 MB, and throughput stayed within approximately 10% of the user-space peak. `linux-mcp` is therefore both a working prototype and a design claim: the enforcement gains are bounded in cost and precise in scope.

5.2 Reflection

The most important question clarified by this project was not simply "where should code run?", but "which invariants must remain trustworthy even when a gateway crashes, is misconfigured, or behaves incorrectly?" This reframing shaped the entire design: rather than asking how to make the gateway more secure, the project asked which properties the gateway should not be solely responsible for enforcing.

From a technical standpoint, the project required working across layers that are often studied separately: implementing a Linux kernel module, using Generic Netlink as a structured kernel–user-space control interface, exporting live state through `sysfs`, and coordinating these pieces with a manifest-aware daemon and UDS tool services. This process deepened understanding of interface ownership, failure handling, recovery logic, and the practical cost of crossing privilege boundaries. A central lesson of systems work was reinforced: the harder problem is often not adding mechanisms, but deciding what each layer should and should not be responsible for.

A second lesson concerned research discipline. Early iterations of this project were formulated around broad claims such as "putting security in the kernel" or "making agents safer". Such claims lack sufficient specificity to be empirically testable. The project became considerably stronger once the problem was narrowed to four concrete control-plane properties (semantic identity binding, approval validity, hash consistency, and durable observability), at which point

the threat model, implementation choices, and evaluation criteria aligned naturally. A systems argument becomes credible only when the claimed benefit is specific enough to be tested directly.

The project also reinforced the value of restraint in privileged-system design. The temptation to move more logic into the kernel was real: every additional check the kernel performs is one the gateway cannot bypass. But manifest parsing, schema validation, and tool execution all belong in user space, where iteration is fast and debugging does not require a VM reboot. A privileged component that does too much is harder to reason about and harder to trust—the small kernel role here was not a limitation but a feature.

One practical consequence of this project is an appreciation of how much the framing of a security claim matters. `linux-mcp` improves a specific part of the control plane; it does not address unsafe tool semantics, poor policy choices, or compromise outside the assumed threat model. Getting the scope right was not just a methodological exercise: a system that claims to solve the full AI-agent security problem is harder to evaluate, harder to trust, and ultimately harder to deploy than one that solves a precise sub-problem well. The hardest part was resisting the temptation to widen the scope as the implementation took shape.

The broader lesson is simply that better security is not always achieved by adding more machinery. Moving a small set of authority-bearing state transitions into a stronger enforcement domain turned out to be more tractable, and arguably more useful, than attempting a fuller redesign.

5.3 Further Work

Several directions follow naturally from the current prototype.

First, the current kernel arbitration logic is intentionally minimal. It verifies tool identity, agent identity, semantic-hash consistency, and approval-ticket validity, but it does not implement a richer policy framework. A natural next step is to support more expressive admission policies, such as per-tool permissions, role-scoped access control, rate limits (e.g., N requests per minute per agent), or execution budgets (e.g., total time per session). The design principle should be maintained: the kernel verifies policy membership but does not compute policy decisions, which remain in user space, preserving the narrow-kernel invariant.

Second, the approval mechanism can be developed into a more complete governance path. The present design supports defer-and-decide control for sensitive operations, but real deployments may require approval delegation, bounded approval scope, revocation, expiry policies, and richer audit trails linking each decision to a concrete execution context. Strengthening this path would make `linux-mcp` more suitable for environments with accountability or compliance requirements.

Third, recovery behaviour can be improved further. The current prototype already preserves kernel-held registry and approval state across daemon failure, but reconciliation is deliberately

conservative and session-level state is rebuilt after restart. A more mature design could provide more systematic restart handling, richer recovery metadata, and clearer support for interrupted or partially completed requests. This becomes particularly important for long-running agent workflows, where failure and restart are routine rather than exceptional.

Fourth, the evaluation can be extended substantially. Future work should include bare-metal evaluation to eliminate hypervisor timing noise, concurrency levels well beyond the current maximum of 100 concurrent requests (e.g., 500 or 1000), tool registries an order of magnitude larger than those used here, and heterogeneous tool workloads with variable or network-dependent execution times. p_{95} latency is characterised in Chapter 3.12; p_{99} under sustained concurrency remains unmeasured and is a natural next step. Comparison against additional hardened user-space designs would also help sharpen the argument by separating the benefits of kernel placement from those achievable through improved user-space engineering alone.

Finally, this work opens a broader research question: which other agent-oriented abstractions deserve operating-system support? `linux-mcp` shows that a small but consequential part of the MCP control plane benefits from OS-level support without requiring AI agents to become first-class OS citizens or requiring a new OS substrate. This project focuses on admission control for tool invocation, but similar arguments may extend to execution budgets, provenance across tool chains, durable approval logs, or agent-scoped resource accounting. Exploring these directions would help clarify whether support for AI agents should be understood as entirely new operating-system substrates, or as conventional systems extended with a small number of carefully scoped, agent-aware mechanisms. In that sense, the contribution of `linux-mcp` is best read as a precise first step rather than a complete answer.

References

[1] Yao S, Zhao J, Yu D, et al. ReAct: Synergizing reasoning and acting in language models. arXiv preprint arXiv:221003629. 2023.

[2] Liu X, Yu H, Zhang H, et al. AgentBench: Evaluating LLMs as agents. arXiv preprint arXiv:230803688. 2024.

[3] Schick T, Dwivedi-Yu J, et al. Toolformer: Language models can teach themselves to use tools. arXiv preprint arXiv:230204761. 2023.

[4] Park JS, O'Brien J, Cai CJ, Morris MR, Liang P, Bernstein MS. Generative Agents: Interactive Simulacra of Human Behavior. In: UIST; 2023. .

[5] Qin Y, Liang S, Ye Y, et al. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. arXiv preprint arXiv:230716789. 2024.

[6] Mei K, et al. AIOS: LLM Agent Operating System. arXiv preprint arXiv:240316971. 2024.

[7] Liu R, Zhe T, Wang D, et al. AgentOS: From Application Silos to a Natural Language-Driven Data Ecosystem. arXiv preprint arXiv:260308938. 2026.

[8] Packer C, Fang V, Patil SG, Lin K, Wooders S, Gonzalez JE. MemGPT: Towards LLMs as Operating Systems. arXiv preprint arXiv:231008560. 2023.

[9] Steinberger P. OpenClaw: Open-source Autonomous AI Agent; 2026. `https://openclaw.ai`.

[10] Peter S, Li J, Zhang I, et al. Arrakis: The operating system is the control plane. In: OSDI; 2014. .

[11] Greshake K, et al. Not what you've signed up for: Compromising real-world LLM-integrated applications with Indirect Prompt Injection. arXiv preprint arXiv:230212173. 2023.

[12] HKUDS. OpenHarness: Open Agent Harness with a Built-in Personal Agent—Ohmo!; 2026. Initial open-source release v0.1.0: April 1, 2026. `https://github.com/HKUDS/OpenHarness`.

[13] Tanenbaum AS, Bos H. Modern Operating Systems. 5th ed. Pearson; 2022.

[14] Chase H. LangChain; 2022. Open-source LLM application framework. `https://github.com/langchain-ai/langchain`.

[15] LangChain AI. LangGraph: A Framework for Building Stateful Multi-Actor LLM Applications; 2024. Open-source agent orchestration framework. https://github.com/langchain-ai/langgraph.

[16] Gravitas S. AutoGPT: An Autonomous GPT-4 Experiment; 2023. Open-source autonomous agent framework. https://github.com/Significant-Gravitas/AutoGPT.

[17] CrewAI Inc . CrewAI: Framework for Orchestrating Role-Playing Autonomous AI Agents; 2024. Open-source multi-agent collaboration platform. https://crewai.com.

[18] Microsoft Corporation. Semantic Kernel: Open-Source SDK for LLM Integration; 2023. Released publicly March 2023; reached v1.0 in early 2024. https://github.com/microsoft/semantic-kernel.

[19] Wu Z, et al. OS-Copilot: Towards Generalist Computer Agents with Self-Improvement. arXiv preprint arXiv:240207456. 2024.

[20] Harness Inc . Harness AI DevOps Agent; 2025. AI-powered autonomous workers embedded in governed CI/CD pipelines. https://developer.harness.io/docs/platform/harness-aida/ai-devops/.

[21] Apple Inc . Private Cloud Compute: A new frontier for AI privacy in the cloud; 2024. https://security.apple.com/blog/private-cloud-compute/.

[22] Model Context Protocol. Architecture Specification; 2025. https://modelcontextprotocol.io/specification/2025-06-18/architecture.

[23] Patil SG, Zhang T, Wang X, Gonzalez JE. Gorilla: Large Language Model Connected with Massive APIs; 2023.

[24] Liu Y, et al. Prompt Injection Attack against LLM-integrated Applications. arXiv preprint arXiv:230605499. 2023.

[25] Wang Z, Tu H, Zhang L, et al. Your Agent, Their Asset: A Real-World Safety Analysis of OpenClaw. arXiv preprint arXiv:260404759. 2026.

[26] Wang Y, Gao H, et al. A Systematic Security Evaluation of OpenClaw and Its Variants. arXiv preprint arXiv:260403131. 2026.

[27] Wang G, Xie Y, Jiang Y, Mandlekar A, Xiao C, Zhu Y, et al.. Voyager: An Open-Ended Embodied Agent with Large Language Models; 2023.

[28] Yang J, Jimenez CE, Wettig A, Lieret K, Yao S, Narasimhan K, et al.. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering; 2024.

[29] Lim SY, Agrawal S, Han X, Eyers D, O'Keeffe D, Pasquier T. Securing Monolithic Kernels using Compartmentalization. arXiv preprint arXiv:240408716. 2024.

[30] Schneider M, Masti RJ, Shinde S, Capkun S, Perez R. SoK: Hardware-supported Trusted Execution Environments. arXiv preprint arXiv:220512742. 2022.

[31] Lefeuvre H, Dautenhahn N, Chisnall D, Olivier P. SoK: Software Compartmentalization. arXiv preprint arXiv:241008434. 2024.

[32] Ji Z, Wu D, Jiang W, Ma P, Li Z, Gao Y, et al. Taming Various Privilege Escalation in LLM-Based Agent Systems: A Mandatory Access Control Framework. arXiv preprint arXiv:260111893. 2026.

[33] Shi T, He J, Wang Z, Wu L, Li H, Guo W, et al. Progent: Programmable Privilege Control for LLM Agents. arXiv preprint arXiv:250411703. 2025.

[34] Linux Kernel Documentation. Seccomp BPF; 2026. `https://docs.kernel.org/userspace-api/seccomp_filter.html`.

[35] Linux Kernel Documentation. Linux Security Modules (LSM); 2026. `https://docs.kernel.org/admin-guide/LSM/index.html`.

[36] Linux Kernel Documentation. AppArmor; 2026. `https://docs.kernel.org/admin-guide/LSM/apparmor.html`.

[37] Linux Kernel Documentation. SELinux; 2026. `https://docs.kernel.org/admin-guide/LSM/SELinux.html`.

[38] Linux Kernel Documentation. Landlock; 2026. `https://docs.kernel.org/userspace-api/landlock.html`.

[39] Brimhall B, Garrard J, de la Garza C, Coffman J. A Comparative Analysis of Linux Mandatory Access Control Policy Enforcement Mechanisms. In: Proceedings of the 16th European Workshop on System Security (EuroSec); 2023. .

[40] deepset GmbH. Haystack: Open-Source LLM Orchestration Framework; 2023. Open-source AI pipeline and agent framework. `https://haystack.deepset.ai`.

[41] Chen Y, Reymondjohnson S, Sun Z, Lu L. Shreds: Fine-grained Execution Units with Private Memory. In: Proceedings of the IEEE Symposium on Security and Privacy (S&P); 2016. p. 56-71.

[42] Stevens WR, Rago SA. Advanced Programming in the UNIX Environment. 3rd ed. Addison-Wesley; 2013.

[43] Bishop M. Computer Security: Art and Science. Addison-Wesley; 2003.

[44] Lee Y, Min C, Lee B. ExpRace: Exploiting Kernel Races through Raising Interrupts. In: Proceedings of the 30th USENIX Security Symposium. Vancouver, BC; 2021. p. 2363-80.

[45] Klein G, Andronick J, Elphinstone K, Heiser G, Cock D, Derrin P, et al. seL4: Formal Verification of an Operating-System Kernel. Communications of the ACM. 2010;53(6):107-15.

[46] Linux Kernel Documentation. Netlink Socket Family; 2026. `https://docs.kernel.org/userspace-api/netlink/intro.html`.

[47] West RM. Best practice in statistics: Use the Welch t-test when testing the difference between two groups. Annals of Clinical Biochemistry. 2021;58(4):267-9.

Acknowledgment

Give your acknowledgment to people who helped you during the project here. Maximum length of this section is 1 page. You may thank your supervisor but DO NOT MENTION YOUR SUPERVISOR’S NAME HERE due to the blind marking policy.

Appendices

Disclaimer

This report is submitted as part requirement for the undergraduate degree programme at Queen Mary University of London, and Beijing University of Posts and Telecommunications. It is the product of my own labour except where indicated in the text. The report may be freely copied and distributed provided the source is acknowledged.

BUPT No.: 2022213192

QMUL No.: 221166471

Full Name (Pin Yin): Li Xiheng

Full Name (Chinese): 李溪恒

Signature:

Date:12-04-2026

Acknowledgement of using Generative AI

I acknowledge the use of the following Generative AI tools:

Publisher	**Name, version, URL**	**Use**	**Section**
OpenAI	Codex (GPT-5-based coding assistant) `https://openai.com/` ChatGPT (GPT-5.4) `https://chatgpt.com/`	Used for translating support during report preparation.	Chapter 2; Chapter 4; Part of the source code
Anthropic	Claude Code (claude-sonnet-4-6) `https://claude.ai/code`	Used for grammar editing, and academic writing support during report preparation. Also, part of the scripts and source code are done by vibe-coding using Claude Code.	Chapter 2; Chapter 4; Part of the source code

I confirm the submitted work is my own work.

Project specification

北京邮电大学 本科毕业设计（论文）任务书

Project Specification Form

Part 1 - Supervisor

论文题目 Project Title	From Traditional Operating Systems to AI Agent Operating Systems: Exploring AI-Native OS through the Lens of the MCP Protocol		
题目分类 Scope	Research	Software Development	Software
主要内容 Project description	As AI agents increasingly function as autonomous computing entities capable of learning, reasoning, and collaboration, the fundamental limitations of traditional operating systems (OS) have become apparent, creating a need for a new generation of agent-oriented OS. This thesis focuses on developing a prototype of an agent-native OS centered around the Model Context Protocol (MCP). Specifically, it proposes an MCP-enabled kernel module that provides unified interfaces for AI agents to register, exchange contextual information, and coordinate computational tasks. Furthermore, it investigates how MCP-based kernel-level scheduling mechanisms can manage computational resources among multiple agents to ensure fairness, efficiency, and security. The results aim to lay the groundwork for future operating systems in which AI agents act as first-class citizens within the computing environment.		
关键词 Keywords	AI Agent, MCP, kernel		
主要任务 Main tasks	1 Analyze the evolution of OS architectures from process-centric to agent-centric paradigms and identify the key design principles of agent-native OS.		
	2 Study the internal structure of the MCP protocol and design how to provide its functionalities from the kernel level.		
	3 Implement a MCP-enabled kernel module and design the exposed APIs.		
	4 Evaluate the performance and scalability of the MCP-enabled kernel model, and discuss how it achieves efficiency, fairness, and security.		
主要成果 Measurable outcomes	1 A comprehensive review chapter comparing traditional OS abstractions with emerging requirements on OS from AI agents.		
	2 A detailed design discusses how MCP interfaces can be mapped to system calls or other interfaces the kernel exposes to user-level applications, and how MCP functionalities are implemented in the kernel.		
	3 Implement a working prototype of the MCP-enable kernel module, along with API documentation describing the exposed interfaces.		

北京邮电大学 本科毕业设计（论文）任务书

Project Specification Form

Part 2 - Student

学院 School	International School	**专业 Programme**	**Telecommunications Engineering with Management**		
姓 Family name	Li	**名 First Name**	Xiheng		
BUPT 学号 BUPT number	2022213192	**QM 学号 QM number**	221166471	**班级 Class**	2022215107
论文题目 Project Title	From Traditional Operating Systems to AI Agent Operating Systems: Exploring AI-Native OS through the Lens of the MCP Protocol				
论文概述 Project outline **Write about 500-800 words** **Please refer to Project Student Handbook section 3.2**	This project aims to explore the design of an AI-Native Operating System (AI-Native OS) based on the Model Context Protocol (MCP). The system integrates a kernel-level MCP module, enabling native agent registration, context exchange, and scheduling. Through the integration of these layers, the project seeks to build a secure, efficient, and extensible foundation for next-generation operating systems where AI agents are treated as first-class computing entities. This project bridges MCP-based agent systems and next-generation operating system design, integrating secure automation with kernel-level scheduling. It aims to establish a secure and scalable foundation for future AI-native computing environments. The project addresses the needs of OS researchers and AI developers who require: 1. Unified interfaces for agent registration and context sharing; 2. Extensible OS infrastructure for managing heterogeneous agents. The developer feedback regarding system usability, safety, and performance. The project adopts a bottom-up system design combining kernel engineering and AI-driven automation. Key methodologies include: 1. Implementation of agents registration, context sharing, and cooperative scheduling. 2. Scheduling algorithm to balance fairness and resource efficiency using contextual metadata. The experiments will test the following hypotheses: 1. MCP-based kernel scheduling enhances fairness and efficiency compared with traditional schedulers. 2. The integrated kernel–server architecture provides lower latency and higher scalability than user-space MCP implementations. 3. Performance metrics include latency, CPU utilization, security violation rate, and scalability under multi-agent workloads. The experiments will be conducted in the following environment: 1. Programming Languages: TBD (kernel module), Python (testing and generation) 2. Software Tools: GCC, Clang, Docker, GDB, FastMCP, LangChain 3. Operating Environment: Linux Kernel 6.x (Ubuntu 22.04) 4. Hardware: x86-64 workstation or virtual machine				

	Background materials Zhang X, Zhang Y. Planet as a Brain: Towards Internet of AgentSites based on AIOS Server. arXiv preprint arXiv:2504.14411; 2025. Available from: https://arxiv.org/abs/2504.14411 Hu X, Xiong T, Yi B, et al. OS Agents: A Survey on MLLM-based Agents for General Computing Devices Use. arXiv preprint arXiv:2508.04482; 2025. Available from: https://arxiv.org/abs/2508.04482 Koubaa A. Agent Operating Systems (Agent-OS): A Blueprint Architecture for Real-Time, Secure and Scalable AI Agents. TechRxiv; 2025. Available from: https://www.techrxiv.org/users/689908/articles/1330306 Mastouri M, Ksontini E, Kessentini W. Making REST APIs Agent-Ready: From OpenAPI to MCP Servers for Tool-Augmented LLMs. arXiv [Preprint]. 2025;arXiv:2507.16044. Available from: https://arxiv.org/abs/2507.16044 Widjaja F, Chen Z, Zhou J. BioinfoMCP: A Unified Platform Enabling MCP Interfaces in Agentic Bioinformatics. arXiv [Preprint]. 2025;arXiv:2510.02139. Available from: https://arxiv.org/abs/2510.02139 Wang Z, Chang Q, Patel H, Biju S, Wu CE, Liu Q, et al. MCP-Bench: Benchmarking Tool-Using LLM Agents with Complex Real-World Tasks via MCP Servers. arXiv [Preprint]. 2025;arXiv:2508.20453. Available from: https://arxiv.org/abs/2508.20453 Tanenbaum AS, Bos H. Modern Operating Systems. 5th ed. Pearson; 2022. OpenAI. Model Context Protocol (MCP) Specification. 2024. Available from: https://github.com/modelcontextprotocol OpenAI Research. MCP Security Framework and Safe Context Exchange. arXiv preprint; 2025. Almeida R. On the Stability of Memory-Dependent Multi-Agent Systems. Future Generation Computer Systems. 2025 (In Press). Available from: https://www.sciencedirect.com/science/article/pii/S0016003224008238
道德规范 **Ethics**	Please confirm by checking the box: ☒ I confirm that I have discussed ethical issues with my supervisor.

Please discuss ethical issues with your supervisor. **Please refer to Project Student Handbook section 4.1 and Project Ethics Guidance**	Summary of ethical issues: This project does not involve any ethical issues.
中期目标 **Mid-term target.** **It must be tangible outcomes, E.g. software, hardware or simulation.** **It will be assessed at the mid-term oral.**	A working prototype demonstrating preliminary kernel-level MCP module. Deliverables include: 1. A thorough survey about the evolution from traditional process-centric OS to agent-centric AI-native OS 2. Kernel module implementing agent registration & context exchange;

Work Plan (Gantt Chart)

Fill in the sub-tasks and insert a letter X in the cells to show the extent of each task

	Nov 1-15	Nov 16-30	Dec 1-15	Dec 16-31	Jan 1-15	Jan 16-31	Feb 1-15	Feb 16-28	Mar 1-15	Mar 16-31	Apr 1-15	Apr 16-30
Task 1 Analyze the evolution of OS architectures from process-centric to agent-centric paradigms and identify the key design principles of agent-native OS.												
Review traditional OS architectures	X	X	X									
Analyse agent computing requirements, context management, and fairness	X	X	X									
Compare traditional OS abstractions vs AI-native demands			X	X	X							
Formulate design principles and theoretical framework for AI-Native OS			X	X	X							
Task 2 Study the internal structure of the MCP protocol and design how to provide its functionalities from the kernel level.												
Analyse structure, context flow, and core mechanisms of MCP.	X	X	X									
Map MCP functions to kernel system calls			X	X	X							
Define kernel-side data structures (context tables, registry).					X	X	X					
Write design specification and interface documentation.					X	X	X					
Task 3 Implement a MCP-enabled kernel module and design the exposed APIs.												
Implement a lightweight user-space daemon that loads the MCP manifest, manages tools.					X	X	X					
Build the tool adapter layer for external tools.					X	X	X					
Achieve working prototype and demo integration workflow.					X	X	X					
Build an automated test runner that compares MCP tool outputs with baseline executions to verify correctness.							X	X	X			
Task 4 Evaluate the performance and scalability of the MCP-enabled kernel model, and discuss how it achieves efficiency, fairness, and security.												
Define performance metrics and design experiments					X	X	X					
Conduct performance & scalability experiments under multi-agent workloads.							X	X	X	X		
Run static & runtime security tests							X	X	X	X		

Summarize results, visualize data, evaluate the result									X	X	X	X

Early-term progress report

北京邮电大学 本科毕业设计（论文）初期进度报告

Project Early-term Progress Report

学院 **School**	International School	专业 **Programme**	**Telecommunications Engineering with Management**		
姓 **Family name**	Li	名 **First Name**	Xiheng		
BUPT 学号 BUPT number	2022213192	**QM 学号 QM number**	221166471	班级 **Class**	2022215107
论文题目 **Project Title**	From Traditional Operating Systems to AI Agent Operating Systems: Exploring AI-Native OS through the Lens of the MCP Protocol				

1. Introduction

The integration of Large Language Models (LLMs) into operating systems has catalyzed a paradigm shift from static instruction execution to dynamic, autonomous agent behavior. However, current frameworks operate atop general-purpose operating systems that lack native abstractions for managing the lifecycle of LLM inference. Relying solely on traditional process scheduling results in uncontrolled contention for the limited context window and lacks isolation mechanisms to prevent agents from monopolizing system resources or accessing unauthorized memory regions.

This project proposes an AI-Native Operating System architecture that integrates the Model Context Protocol (MCP) [1] directly into the kernel subsystem. By elevating "Agent Context" and "Tools" to first-class kernel citizens, we aim to replace passive user-space management with active kernel-level scheduling and security enforcement, thereby bridging the gap between traditional OS resource management and autonomous agent workflows.

2. Literature Review

2.1 Limitations of User-Space Agent Frameworks

Current agent architectures, such as AutoGPT [2] and Generative Agents [3], operate as user-space processes atop general-purpose operating systems. However, these traditional OS environments lack native abstractions for LLM-centric workflows. Qin et al. [4] pointed out that existing frameworks lack a unified mechanism to manage LLM resource contention. Instead of scheduled access, agents employ an inefficient "trial-and-error" approach: they

attempt to load prompts into GPU memory until hitting hardware limits (e.g., CUDA out-of-memory exceptions), forcing expensive deallocation and multiple retry attempts that severely degrade system throughput.

Furthermore, current general-purpose OS abstractions treat Agent Context as opaque data blobs, forcing frameworks to manage state retrieval sequentially in user-space, ignoring potential data-level parallelism. Lacking native parallel primitives for "Agent Context," the system fails to leverage multi-core parallelism for concurrent reasoning tasks, forcing agents to process information strictly linearly.

2.2 The "LLM as OS" Metaphor vs. Reality

Recent works like AIOS [5] , MemGPT[6] , and Xserve [7] have popularized the concept of an "OS for LLMs." MemGPT introduces a virtual memory hierarchy to manage limited context windows, Xserve introduces a system-level abstraction and service, which utilizes a virtual capability layer with a unified API to streamline the integration of AI into applications, while AIOS proposes an LLM-centric scheduler that isolates resources and services into an AIOS kernel to improve scheduling, context management, and resource utilization efficiency. However, a critical analysis reveals fundamental architectural limitations in these systems, the MemGPT focus mainly on context memory and text generation, which is not enough for an AI agent operating system. While AIOS demonstrates improved throughput across various agent benchmarks, it operates entirely as a user-space application (running on the Python interpreter). Consequently, it lacks strict isolation: its 'Access Manager' relies on high-level logic checks, meaning a malicious agent could potentially crash the host process or bypass restrictions. Furthermore, AIOS relies on software-level context interruption rather than true hardware preemptive scheduling of GPU resources, preventing it from enforcing hard real-time guarantees for mission-critical agent responses.

Also, in the security level, the existing AI operating systems lack isolation. A malicious agent can easily crash the host Python process or access unauthorized memory, as demonstrated in recent prompt-injection attacks on agentic systems [8][9].

Some mobile operating systems have begun integrating machine learning workloads directly into the system infrastructure beyond simple application-level and configured out some solutions to tackle the above problems.

In 2024, Apple introduced a system-wide intelligence layer deeply integrated into iOS and macOS[10]. A key component of its architecture is the combination of App Intents and the Semantic Index. App Intents is a Swift-based protocol that redefines application functionalities using a standardized intent schema and exposes them to the operating system, enabling the LLM to invoke applications to fulfill user instructions. The Semantic Index organizes user data across applications to provide context-aware capabilities. Apple adopts a hybrid execution model, processing sensitive tasks on device while offloading more computationally intensive tasks to Private Cloud Compute.

Google introduced Private Compute Core to bring a new layer of privacy to the Android ecosystem. for OS-level ML features [11]. Private Compute Core is an open source, secure environment that is isolated from the rest of the operating system and apps. It ensures that sensitive data processed by ML models stays isolated from the network via strict API boundaries, communicating with the cloud only through Private Compute Services for model updates [12].

While these industrial solutions validate the necessity of OS-level AI integration, they are proprietary and siloed ecosystems. They lack a unified, open interface for third-party agents to interact with the kernel's context management. This project differs by proposing an open-source, kernel-native implementation using the standardized MCP, aiming to democratize system-level agent capabilities for the Linux ecosystem.

2.3 Kernel-Level Opportunities and the MCP Protocol

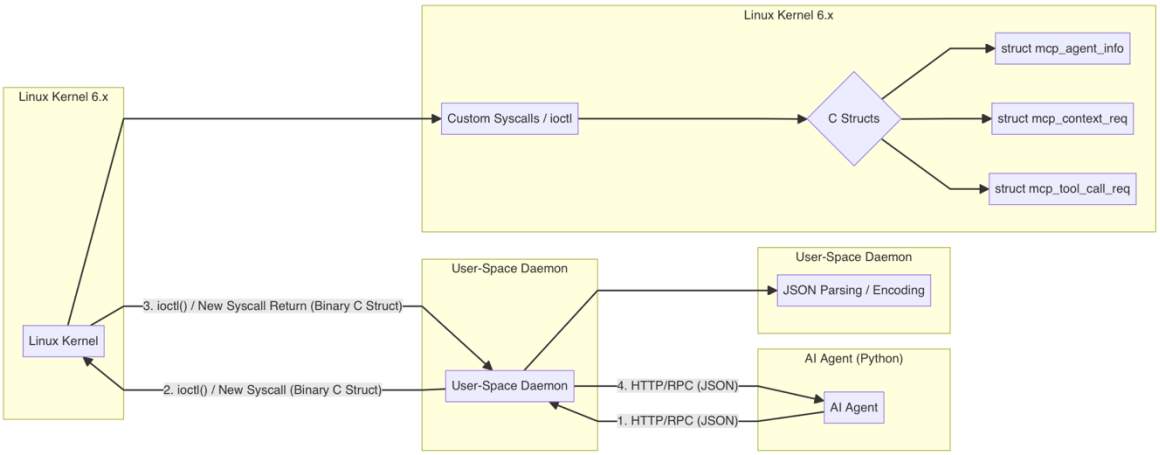
The MCP provides a standardized schema for linking agents to data, and it can effectively prevent the malicious instructions sent by agents. However, current MCP implementations are exclusively high-level (TypeScript/Python). There is a distinct lack of research on implementing MCP primitives as System Calls. This project fills this gap by exploring a Kernel-Level MCP Module to offer lower latency and mandatory access control.

3. Project Progress Summary

3.1 Completed Work

In accordance with Task 1 and the early stages of Task 2 in the project work plan, the following progress has been made:

- Theoretical Framework Analysis: I have completed a comparative analysis between traditional process-centric paradigms (Process Control Blocks - PCB) and agent-centric demands (Agent Context Blocks). I determined that the Linux task_struct needs to be extended or pointed to by a new auxiliary structure to hold MCP metadata.
- MCP Protocol Analysis: I have deeply analyzed the MCP specification and security, specifically the lifecycle of initialization and the prompts, resources, and tools primitives. I have drafted a mapping table that translates MCP JSON-RPC messages into theoretical C-language kernel structures.



- Environment Setup: A development environment has been successfully established on my MacBook using VMware, the Linux version is Ubuntu 22.04 LTS.
    - Compiled Linux Kernel 6.x source code to ensure the environment is ready for custom module insertion.
    - Installed and tested FastMCP to understand the standard user-space behavior of the MCP protocol.
    - Configured QEMU/KVM for safe kernel debugging (GDB) to prevent crashing the host machine during development.

In the future, I will deploy the system on a high-performance server to evaluate scalability and accurate performance metrics under multi-agent workloads

3.2 Comparison with Work Plan

The project is currently on schedule.

- Planned: Analyze OS architectures and MCP internal structure by mid-January.
- Actual: The literature review and architectural analysis are complete. The mapping of MCP functions to kernel system calls (Task 2) is currently in progress.
- Adjustment: The complexity of parsing JSON (used by MCP) inside the kernel was identified as a technical risk. I have decided to implement a lightweight binary serialization protocol for the kernel boundary, with a user-space daemon handling the JSON conversion, which slightly refines the approach in Task 3.

4. References

[1]. Model Context Protocol, "Model Context Protocol Specification," 2024. [Online]. Available: https://spec.modelcontextprotocol.io/

[2]. T. Richards, "Auto-GPT: An Autonomous GPT-4 Experiment," GitHub Repository, 2023. [Online]. Available: https://github.com/Significant-Gravitas/Auto-GPT

[3]. J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative Agents: Interactive Simulacra of Human Behavior," in Proc. ACM Symp. User Interface Softw. Technol. (UIST), 2023.

[4]. Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. Toolllm: Facilitating large language models to master 16000+ real-world apis. ICLR, 2024.

[5]. Kai Mei et al., "AIOS: LLM Agent Operating System," arXiv preprint arXiv:2403.16971, 2024.

[6]. C. Packer, V. Fang, S. G. Patil, K. Lin, S. Wooders, and J. Gonzalez, "MemGPT: Towards LLMs as Operating Systems," arXiv preprint arXiv:2310.08560, 2024.

[7]. Jinrong Yang, Zimeng Wang, Rong Chen, and Haibo Chen. 2025. A System-level Abstraction and Service for Flourishing AI-powered Applications. In Proceedings of the 16th ACM SIGOPS Asia-Pacific Workshop on Systems (APSys '25). Association

for Computing Machinery, New York, NY, USA, 106–114. https://doi.org/10.1145/3725783.3764406

[8]. OWASP GenAI Security Project, "LLM01: Prompt Injection," OWASP Top 10 for Large Language Model Applications, 2025. [Online]. Available: https://genai.owasp.org/llmrisk/llm01-prompt-injection/

[9]. F. Liu et al., "Prompt Injection attack against LLM-integrated Applications," arXiv preprint arXiv:2306.05499, 2023.

[10]. Apple Inc., "Apple Intelligence: System Architecture and Privacy," Apple Platform Security Guide, 2024. [Online]. Available: https://security.apple.com/documentation/

[11]. Google. Android Private Compute Core: Technical Whitepaper. Google Open Source Security, 2022. [Online]. Available: https://services.google.com/fh/files/misc/android-security-paper-2024.pdf

[12]. Google. Private Compute Services. GitHub Repository, 2022. [Online]. Available: https://github.com/google/private-compute-services

是否符合进度？On schedule as per GANTT chart?
YES

下一步 Next steps:

Kernel Data Structure Definition: Define the struct mcp_agent and struct mcp_context in C code within the kernel module to store agent metadata.

Kernel Module Skeleton: Write the initial Character Device Driver (/dev/mcp_core) that allows user-space processes to "register" as agents via ioctl calls.

Prototype Integration: Develop a minimal user-space Python script using ctypes to interact with the kernel module, demonstrating basic agent registration and context retrieval.

Mid-term progress report

北京邮电大学 本科毕业设计（论文）中期进度报告

Project Mid-term Progress Report

学院 School	International School	**专业 Programme**	**Telecommunications Engineering with Management**		
姓 Family name	Li	**名 First Name**	Xiheng		
BUPT 学号 BUPT number	2022213192	**QM 学号 QM number**	221166471	**班级 Class**	2022215107
论文题目 Project Title	From Traditional Operating Systems to AI Agent Operating Systems: Exploring AI-Native OS through the Lens of the MCP Protocol				
是否完成任务书中所定的中期目标？Targets met (as set in the Specification)? YES					

已完成工作 Finished work:

1. Survey Results

This study first conducts a comprehensive survey on the evolution of operating system architectures, systematically analyzing the paradigm shift from traditional process-centric models to agent-centric AI-native operating systems. Specifically, this survey identifies a structural transition across four core dimensions:

Shift in Computational Abstraction: The system's first-class citizen evolves from the "process" to the "agent," shifting the computing paradigm from deterministic execution to LLM-driven, intent-oriented generation.

Reconstruction of Resource Management: Bottlenecks transition from traditional memory allocation to LLM context windows, token consumption, and API quotas, requiring novel multi-agent scheduling algorithms.

Semanticization of IPC: Inter-module communication shifts from low-level byte streams to highly semantic interactions, utilizing JSON schemas and standardized protocols like the Model Context Protocol to route data between agents and tools.

Redefining Security Boundaries: To mitigate risks like logical privilege escalation caused by model hallucinations, system security must upgrade from hardware-based memory isolation to semantic validation and capability hash-binding.

Against this evolutionary backdrop—particularly the shift towards semantic IPC and the need for redefined security boundaries—this work further investigates the internal structure of MCP and examines how tool invocation is handled in current AIOS prototypes and multi-agent orchestration environments.

A preliminary technical analysis of these existing environments reveals three major limitations. First, tool execution is typically governed purely in user space, making it difficult to enforce strict admission control at the operating-system level. Second, while logging may exist, structured auditing and accounting are rarely maintained as kernel-backed registries with explicit counters and completion tracking. Third, fairness under multi-agent contention is rarely modeled explicitly; resource contention is usually handled heuristically with limited quantitative evaluation.

These findings directly motivated the core design principle of this project: governance must be strictly enforced by a kernel control plane, while semantic parsing and tool execution remain securely isolated in user space.

2. Requirement Analysis

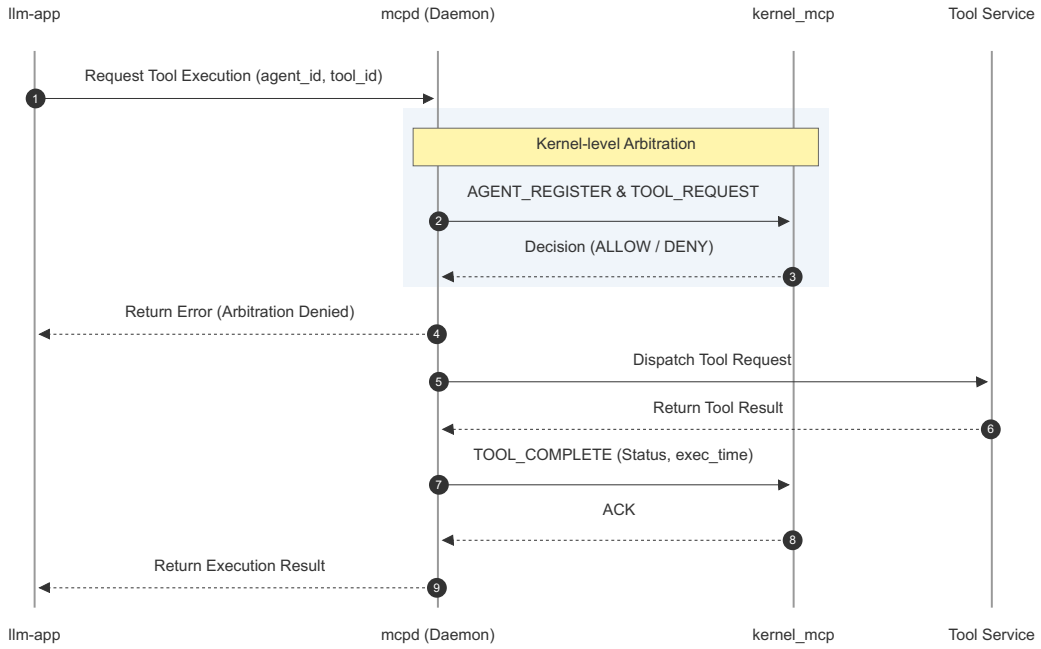
The system requirements are systematically categorized into functional requirements, architectural constraints, and non-functional requirements to ensure robust tool invocation and governance.

Functional Requirements: First, the system must provide explicit kernel-level arbitration. Every tool invocation request must yield a deterministic governance decision from the set {ALLOW, DENY, DEFER} before execution can proceed. Second, the system must support semantic discovery, enabling LLM-based clients to dynamically retrieve tool descriptions, input schemas, and execution examples for automated tool selection. Third, it must support cross-application tool invocation, requiring tools to be hosted and managed as independent resident services rather than ad-hoc inline scripts.

Architectural Constraints: To minimize kernel complexity and mitigate security risks, the system imposes a strict data-plane constraint: JSON parsing is strictly prohibited inside the kernel. The kernel boundary must only accept structured, fixed-size governance attributes (e.g., tool_id, agent_id, perm, cost, and tool_hash).

Non-Functional Requirements: From a non-functional perspective, the system must enforce strict semantic consistency between user-space application manifests and kernel-side registries to prevent semantic drift or substitution attacks. Additionally, the system must guarantee measurable fairness under concurrent multi-agent workloads, ensuring that resource contention is governed quantitatively rather than heuristically. Finally, the system must provide high runtime observability, exposing governance states, audit counters, and registry data to facilitate monitoring and debugging.

3. System Design



The implemented architecture is organized into four layers: an llm-app client layer, an mcpd gateway layer, a kernel_mcp governance layer, and a resident tool service execution layer. At runtime, the llm-app sends a framed JSON request to mcpd through a centralized Unix Domain Socket. The gateway performs routing using semantic metadata and issues a structured TOOL_REQUEST to the kernel via Generic Netlink, carrying only fixed-size governance attributes (agent_id, tool_id, perm/cost references, tool_hash, and related control fields). The kernel returns an arbitration decision (ALLOW, DENY, or DEFER). When ALLOW is returned, mcpd forwards the full JSON payload to the corresponding application-specific resident service endpoint. After

execution, mcpd reports completion back to the kernel via TOOL_COMPLETE, including status and latency, and returns the result to the client.

This design enforces a strict control-plane/data-plane separation. The kernel implements the control plane and does not parse or buffer unbounded JSON payloads. The data plane is handled entirely in user space: mcpd is the sole gateway that dispatches requests to resident tool services. Under the current threat model, clients do not directly interact with tool-service IPC endpoints; tool services accept requests routed through mcpd, which reduces the risk of bypassing kernel arbitration.

To prevent semantic drift between user-space manifests and kernel-side governance records, the system uses a canonical hash-binding mechanism. The hash is computed over stable capability attributes (tool_id, name, perm, cost, description, input_schema, and examples) while excluding runtime metadata such as endpoint paths and process identifiers. During startup, reconcile_kernel.py performs one-to-one comparison between manifest-defined tool entries and kernel registry entries; any mismatch triggers fail-fast termination. During runtime arbitration, tool_hash consistency checks provide an additional guardrail against executing a tool definition whose semantics diverge from the kernel-approved baseline.

4. Prototype Implementation

While the early-term report proposed using a character device and ioctl for kernel-user communication, the implementation transitioned to Generic Netlink. This adjustment provides a more structured and extensible IPC mechanism for multiplexing tool requests compared to legacy ioctl boundaries. A functional prototype has been implemented across kernel space and user space.

On the kernel side, the kernel_mcp module provides Generic Netlink commands for tool/agent registration, request arbitration, and completion reporting. It maintains in-kernel registries for tools and agents, implements per-agent token buckets for rate governance, and updates audit/accounting counters. These counters and registry snapshots are exposed via sysfs for observability. Basic stability checks were conducted through repeated module load/unload cycles, and no kernel warnings or crashes were observed during these tests.

On the user-space side, the mcpd gateway loads application manifests, computes canonical semantic hashes, performs startup reconciliation, enforces arbitration outcomes, forwards requests to resident tool services, and reports completion back to the kernel. For client-facing discovery, mcpd exposes semantic-only tool fields (descriptions, schemas, and examples) while avoiding leakage of runtime routing details such as service endpoints.

In the execution layer, multiple tools were migrated to persistent resident services over Unix Domain Sockets. A shared service library implements a 4-byte big-endian length-prefixed JSON protocol and supports concurrent handling via a thread-per-connection model. Each tool module supports both a debug mode and a resident service mode for development convenience. Security-oriented validations were added where applicable, including path restriction checks for file-related tools, safe evaluation constraints for expression-based tools, and basic schema/parameter validation.

These manifest snippets below illustrate how linux-mcp separates semantics from governance. The fields description, input_schema, and examples define tool semantics for discovery and selection in the llm-app layer, while runtime routing metadata (mode, endpoint, service_path, app_impl) is consumed by mcpd to dispatch requests to the correct resident tool service. Governance-related attributes form the canonical binding used for kernel-side arbitration and for startup reconciliation, ensuring a strict one-to-one correspondence between user-space manifests and the kernel registry. This design keeps JSON payload parsing and natural-language semantics in user space, while allowing the kernel control plane to enforce admission control and accounting using fixed-size structured identifiers.

```
{
  "app_id": "file_manager_app",
  "app_name": "File Manager App",
```

```
  "mode": "uds_service",
  "endpoint": "/tmp/linux-mcp-apps/file_manager_app.sock",
  "service_path": "tool-app/app_service.py",
  "app_impl": "tool-app/apps/file_manager_app.py",
  "tools": [
    {
      "tool_id": 12,
      "name": "file_delete",
      "perm": 1,
      "cost": 1,
      "handler": "file_delete",
      "description": "Delete a file (or directory with recursive=true) under repository root with safety
checks.",
      "input_schema": {
        "type": "object",
        "required": ["path"],
        "properties": {
            "path": { "type": "string" },
            "recursive": { "type": "boolean" },
            "allow_missing": { "type": "boolean" }
            },
        "additionalProperties": false
      },
      "examples": [
        { "path": "tmp/demo_created_by_mcp.txt" },
        { "payload": { "path": "tmp/old_logs", "recursive": true }}
      ]
    }
  ],
…
}
```


5. Functional Validation and System Integration

At the current stage, experimental validation focused on end-to-end system integration and functional correctness. Build validation confirmed the successful compilation of all kernel modules and C components, as well as the stable execution of the Python-based gateway and tool services. For LLM integration, the llm-app client was successfully connected to the DeepSeek large language model via API. Tests demonstrated that DeepSeek could correctly comprehend the provided tool manifests and generate well-formed JSON tool-invocation requests according to the protocol.

Crucially, the end-to-end closed-loop workflow was successfully verified across multiple different tools. The runtime observations confirmed the following sequence: user inputs trigger LLM responses; the llm-app submits tool requests to the mcpd gateway; the gateway initiates a governance check at the kernel level; the kernel successfully performs arbitration to evaluate the request; upon an ALLOW decision, the gateway dispatches the execution to the correct external tool-app; the tool executes its internal logic properly; and finally, the result is accurately propagated back to the llm-app. This functional testing demonstrates that the framework achieves its primary goal: enabling LLM-driven tool execution while enforcing strict kernel-level scrutiny.

6. Observability and GUI Integration

A graphical demo client was implemented in the llm-app layer to support interactive end-to-end validation. The GUI is built with PySide6 and connects to the gateway exclusively through the

centralized Unix Domain Socket interface exposed by mcpd. It provides an interactive workflow that mirrors the CLI pipeline: the GUI refreshes the application catalog via {"sys":"list_apps"}, retrieves tool catalogs via {"sys":"list_tools"}, and issues execution requests using {"kind":"tool:exec", ...} so that every invocation still goes through kernel arbitration and completion reporting enforced by mcpd .

The GUI exposes two practical observability surfaces during demonstration. First, it visualizes the discovered app/tool catalog after refreshing from mcpd, making it easy to confirm that the client only sees semantic fields and the tool hash, rather than runtime routing details . Second, for each user input, it shows the selected app and tool, the selector source and rationale, the request id, and the execution response including timing (t_ms), which helps trace the full loop in a single screen . The GUI reuses the same selection logic shared with the CLI, allowing consistent behavior across interactive and scripted experiments .

7. Summary

A functional linux-mcp prototype has been successfully implemented and validated, achieving kernel-governed tool invocation while strictly isolating tool semantics and execution within user space. The runtime workflow is systematically orchestrated: the llm-app (CLI/GUI) discovers tools via manifest-provided schemas and submits execution requests exclusively to the mcpd gateway. The gateway then queries the kernel via Generic Netlink to obtain an arbitration decision. Only upon receiving an ALLOW signal does mcpd dispatch the payload to per-app resident UDS services, subsequently reporting a TOOL_COMPLETE status back to the kernel for strict resource accounting. To guarantee system integrity, the design incorporates a fail-fast startup reconciliation mechanism utilizing canonical semantic hashing, which effectively prevents manifest-kernel semantic drift. Furthermore, the architecture is reinforced with kernel-side registries exposed via sysfs for observability, and persistent tool services safeguarded by length-prefixed JSON IPC and rigorous input validation.

尚需完成的任务 Work to do:

First, the arbitration model can be extended to support more granular governance, including hierarchical permission tiers, dynamic cost budgets, and adaptive backoff algorithms for deferred requests.
Second, deeper performance profiling such as syscall-level tracing is planned to rigorously quantify the IPC overhead and benchmark the architecture against traditional subprocess invocations under diverse concurrent workloads.
Finally, system observability will be further improved by expanding sysfs telemetry interfaces and developing standardized, automated test suites to facilitate reproducible evaluations in production environments.

存在问题 Problems:

There are two gaps remain before it can be considered complete and thesis-ready.

The first is security and trust-boundary clarity for high-risk tools (especially file operations). The current design correctly routes execution through a single gateway and uses manifests plus schema constraints, but the defense-grade security story still depends on clearly defined assumptions and systematic edge-case coverage. What is missing is a rigorous threat model (who is trusted, who can connect to which socket, what the attacker can do) and a set of adversarial/edge-case tests that demonstrate the boundary holds under realistic misuse scenarios.

The second is the report still needs stronger quantitative support on performance overhead and correctness under failure modes. A syscall-level comparative analysis would make the overhead of UDS framing and Netlink arbitration and dispatch concrete and comparable to baselines like subprocess invocation or direct syscall workflows.

拟采取的办法 Solutions:

First, to solidify the security and trust boundaries, a rigorous threat model and explicit trust assumptions will be formalized. At the implementation level, the system will enforce a strict, consistent sandboxing mechanism confined to the repository root. This containment strategy will be achieved by actively rejecting absolute paths and directory traversal attempts, as well as mitigating symlink escape vulnerabilities through strict realpath validation and the disabling of symlink resolutions for destructive operations. Additionally, semantic guardrails will be significantly tightened for high-risk actions such as file_delete. The robustness of these architectural boundaries will be empirically validated using targeted adversarial testing and edge-case injection scenarios to simulate realistic misuse.

Second, to strengthen the quantitative evaluation framework, the study will conduct a granular syscall-level overhead analysis utilizing system profiling utilities. This profiling will establish concrete performance metrics that will be directly compared against traditional subprocess-based and direct-invocation baselines. Furthermore, the correctness benchmarking suite will be significantly extended to cover diverse failure modes and exceptional states. This expansion includes validating the fail-fast termination logic during manifest-kernel hash mismatches, evaluating the system's backoff behavior during DEFER arbitration, and testing system resilience against IPC timeouts, partial execution failures, and service restart recovery. To ensure scientific rigor and reproducibility, all benchmarking procedures will be fully automated using standardized scripts designed for structured telemetry and metrics collection.

论文结构 Structure of the final report: (Chapter headings and section sub headings)

Chapter 1: Introduction
Chapter 2: Background
- **2.1 Traditional Operating System**
- **2.2 Existing AI Operating System**
- **2.3 Model Context Protocol**

Chapter 3: Design and Implementation
- **3.1 Overview**
- **3.2 Workflow and Protocols**
- **3.3 Kernel-Side Implementation**
- **3.4 User-Space Implementation**
- **3.5 Safety and Consistency Mechanisms**

Chapter 4: Results and Discussion
- **4.1 Experimental Setup**
- **4.2 Results**
- **4.3 Case Study**
- **4.4 Discussion**

Chapter 5: Conclusion and future work
- **5.1 Conclusion**
- **5.3 Future work**

Supervision log

北京邮电大学 本科毕业设计（论文）教师指导记录表

Project Supervision Log

学院 School	International School	**专业 Programme**	**Telecommunications Engineering with Management**		
姓 Family name	Li	**名 First Name**	Xiheng		
BUPT 学号 BUPT number	2022213192	**QM 学号 QM number**	221166471	**班级 Class**	2022215107
论文题目 Project Title	From Traditional Operating Systems to AI Agent Operating Systems: Exploring AI-Native OS through the Lens of the MCP Protocol				

Please record supervision log using the format below:

Date: dd-mm-yyyy
Supervision type: face-to-face meeting/online meeting/email/messages/other (please specify)
Summary:

Date: 07-11-2025
Supervision type: online meeting
Summary: discussed the project scope and agreed on the overall direction of the linux-mcp prototype, separating kernel governance from user space tool orchestration.

Date: 05-12-2025
Supervision type: online meeting
Summary: reviewed the system architecture, including the Generic Netlink control plane in kernel-mcp and the role of mcpd as the central execution gateway.

Date: 19-12-2025
Supervision type: online meeting
Summary: discussed the interaction flow between the kernel module, mcpd, and the userspace clients, and refined the responsibilities of each layer in the system.

Date: 16-01-2026
Supervision type: online meeting
Summary: discussed the design of manifest-based tool registration and the coordination between tool-app, mcpd, and the kernel registry.

Date: 24-01-2026
Supervision type: online meeting
Summary: discussed the service structure, including resident Unix domain socket services, handler dispatching, and safe file access constraints in the file management tools.

Date: 03-03-2026
Supervision type: online meeting
Summary: discussed the organization of tools into separate apps, including calculator, utility, settings, and file manager modules, to improve maintainability and semantic clarity.

Date: 10-03-2026
Supervision type: online meeting

Summary: discussed the latest progress on semantic hash binding, tool and app discovery in llm-app, and the overall end-to-end workflow from request submission to controlled tool execution.

Additional Appendices

A.1 Wire Protocol Reference

This section provides the complete field specifications and representative implementation fragments for the three inter-component protocols described in Section 3.4.

Complete llm-app to mcpd message reference. The `tool:exec` request carries the tool hash as a 64-character hexadecimal string. If a prior `DEFER` decision returned a `ticket_id`, the client includes it in a replay request:

Listing 2: tool:exec request and response messages (llm-app → mcpd).

```
// tool:exec request
{ "kind": "tool:exec",  "req_id": 42,
  "session_id": "sess-abc", "agent_id": "agent-1",
  "app_id": "sys_app",  "tool_id": 1,
  "tool_hash": "a3f2...c91b",        // 64-char SHA-256 hex
  "payload": {"path": "/tmp"},
  "approval_ticket_id": null }       // set on replay after DEFER

// tool:exec response (ALLOW)
{ "req_id": 42, "status": "ok",
  "result": {"os": "Linux 6.8", "cpu": "4-core"},
  "decision": "ALLOW", "reason": "admitted",
  "ticket_id": 0,  "t_ms": 3 }

// tool:exec response (DEFER)
{ "req_id": 42, "status": "error",
  "result": {}, "error": "deferred pending approval",
  "decision": "DEFER", "reason": "risk_tag:sensitive_read",
  "ticket_id": 7,  "t_ms": 1 }
```

The `approval_reply` message is issued by the client after the user confirms a deferred operation:

Listing 3: approval_reply message (llm-app → mcpd, issued after user confirms a deferred operation).

```
// approval_reply request
{ "sys": "approval_reply",  "session_id": "sess-abc",
  "ticket_id": 7,  "decision": "approve",
  "reason": "user confirmed",  "ttl_ms": 30000 }
```

Complete Netlink attribute encoding. All attributes are 4-byte aligned. String attributes are NUL-terminated and passed as raw bytes. The following Python fragment shows how `mcpd`

encodes a TOOL_REQUEST message:

Listing 4: Python fragment: encoding a TOOL_REQUEST Generic Netlink message (mcpd → kernel).

```
import struct, socket, hashlib

FAMILY_NAME        = b"KERNEL_MCP\x00"
CMD_TOOL_REQUEST   = 10
ATTR_AGENT_ID,   ATTR_TOOL_ID     = 4,  2
ATTR_REQ_ID,     ATTR_TOOL_HASH   = 1,  19
ATTR_AGENT_BINDING, ATTR_AGENT_EPOCH = 28, 29

def build_nla(attr_type, data):
    # NLA header: 2-byte length, 2-byte type; 4-byte aligned
    hdr = struct.pack("HH", len(data) + 4, attr_type)
    pad = (4 - len(data) % 4) % 4
    return hdr + data + b"\x00" * pad

def encode_tool_request(agent_id, tool_id, req_id,
                        tool_hash, binding, epoch):
    attrs = (
        build_nla(ATTR_AGENT_ID,       agent_id.encode() + b"\x00") +
        build_nla(ATTR_TOOL_ID,        struct.pack("=I", tool_id))   +
        build_nla(ATTR_REQ_ID,         struct.pack("=Q", req_id))    +
        build_nla(ATTR_TOOL_HASH,      tool_hash.encode() + b"\x00") +
        build_nla(ATTR_AGENT_BINDING,  struct.pack("=Q", binding))   +
        build_nla(ATTR_AGENT_EPOCH,    struct.pack("=Q", epoch))
    )
    # Generic Netlink header: cmd (1 B), version (1 B), reserved (2 B)
    genl_hdr = struct.pack("BBH", CMD_TOOL_REQUEST, 1, 0)
    return genl_hdr + attrs
```

Semantic hash computation. The semantic hash used for tool identity is a full 256-bit SHA-256 digest computed over a canonical JSON serialisation of the following manifest fields: tool_id, name, app_id, app_name, risk_tags, description, input_schema, examples, path_semantics, and approval_policy. The endpoint field is deliberately excluded so that redeployment to a new socket path does not invalidate the kernel registration.

Listing 5: Python fragment: computing the manifest-derived semantic hash used for tool identity.

```
import json, hashlib
```

```python

SEMANTIC_FIELDS = [
    "tool_id", "name", "app_id", "app_name",
    "risk_tags", "description", "input_schema",
    "examples", "path_semantics", "approval_policy",
]

def semantic_hash(tool_manifest: dict) -> str:
    canonical = {f: tool_manifest[f] for f in SEMANTIC_FIELDS}
    blob = json.dumps(canonical,
                      sort_keys=True,
                      separators=(",", ":"),
                      ensure_ascii=True).encode("utf-8")
    return hashlib.sha256(blob).hexdigest()  # 64-char hex;
        never truncated
```

The full 64-character digest is used without truncation. A 32-bit prefix (8 hex characters) would be vulnerable to birthday collisions after approximately 2^{16} distinct tool registrations, which is a realistic size for a production deployment spanning multiple applications.

Field	In hash?	Description
Application-level fields		
`app_id`	✓	Unique application identifier string
`app_name`	✓	Human-readable application name
`transport`	✗	Must be `"uds_rpc"`
`endpoint`	✗	Absolute socket path under `/tmp/linux-mcp-apps/`
Tool-level fields		
`tool_id`	✓	Numeric identifier, unique within deployment
`name`	✓	Tool name presented to the LLM planner
`description`	✓	Natural-language description of tool behaviour
`risk_tags`	✓	Risk labels controlling kernel DEFER (e.g., `"sensitive_read"`)
`input_schema`	✓	JSON Schema object for payload validation
`examples`	✓	Example invocation payloads
`path_semantics`	✓	Path interpretation hints (e.g., `"repo_rel"`)
`approval_policy`	✓	Policy name triggering kernel DEFER; null for unrestricted tools
`operation`	✗	Operation string forwarded to the tool service
`timeout_ms`	✗	Per-invocation execution timeout (operational metadata)

Table 10: Manifest field specification. ✓ = included in the SHA-256 semantic identity hash; ✗ = excluded (operational metadata whose change should not invalidate a tool registration). Fields are grouped by level: application-level fields appear once per `app_id`; tool-level fields appear once per tool entry.

Complete manifest field specification.

A.2 System Comparison Table

System	Control-plane placement	Tool identity	State durability	Trust anchor & security boundary
`linux-mcp` **(this work)**	Kernel module + user-space gateway	SHA-256 of JSON manifest; kernel-verified at every invocation	Approval tickets and registry survive daemon crash (kernel-held)	**Kernel**; enforcement is independent of gateway correctness
`OpenClaw` [9]	User-space gateway (persistent process)	Skill name/version string in gateway registry	Lost on process restart	User-space process; bypassable via logic flaws [25, 26]
Harness Agent [20]	Cloud-hosted pipeline orchestrator	Pipeline step + RBAC connector identity	Cloud-hosted; platform-resilient	SaaS control plane; RBAC-based governance
AIOS / AgentOS [6, 7]	New OS-level agent kernel (proposed)	Agent/tool registration in LLM scheduler	Not evaluated (proposal stage)	New OS substrate; full redesign
LangChain / LangGraph [14, 15]	In-process user-space library	Tool name string; no cryptographic binding	Lost on process crash	Application process; no inter-invocation isolation
Apple Intelligence [21]	On-device OS + Private Cloud Compute	OS capability entitlements	High; OS-persistent services	Hardware attestation + OS entitlements; closed ecosystem

Table 11: Comparison of representative agent systems. The key differentiator for `linux-mcp` is the combination of cryptographic tool identity and kernel-held durable state, both absent from all user-space frameworks.

Risk and Environmental Impact Assessment

Risk and Environmental Impact Assessment

This project investigates `linux-mcp`, a kernel-assisted control-plane architecture for AI tool invocation. Its goal is to improve the safety and observability of tool use in AI-agent systems by separating semantic interpretation and tool execution in user space from admission control and approval handling in the kernel. Because the system spans an LLM-facing application, a user-space gateway, kernel mechanisms, and external tools, it introduces technical, security, and resource-related risks. A formal risk and environmental impact assessment is therefore necessary.

Following the prescribed method, each risk is assigned a likelihood level L and a consequence level C, and the overall score is calculated as $R = L \cdot C$. This assessment considers risks to project completion, risks of harm to systems or organisations, environmental impact, and losses of time or computational resources.

Project and technical risks

A primary technical risk is kernel-level instability. Since `linux-mcp` moves part of the control plane into the Linux kernel, faults in Netlink communication, synchronization, or state management may lead to crashes or inconsistent behaviour. This risk is assigned moderate likelihood ($L = 3$) and major consequence ($C = 4$), giving $R = 12$. The mitigation is to keep the kernel scope narrow and rely on strict interfaces, staged testing, and stress testing.

A second risk is the possibility that the project does not demonstrate a convincing advantage over strong user-space baselines. The value of `linux-mcp` depends not only on correctness but also on showing that kernel placement provides stronger control properties than user-space mediation alone. This risk is assigned likely likelihood ($L = 4$) and very serious consequence ($C = 3$), giving $R = 12$. It should be mitigated by evaluation against both conventional and hardened user-space baselines under representative performance and adversarial workloads.

A third risk is cross-component integration failure. The system includes an LLM-side application, the `mcpd` daemon, the kernel control plane, and tool-side services, and mismatches between these components may create failures that are difficult to isolate. This risk is assigned moderate likelihood ($L = 3$) and serious consequence ($C = 2$), giving $R = 6$. This can be reduced through explicit protocol definitions, schema synchronization, and end-to-end validation.

Security and operational risks

A central operational risk is unsafe or unauthorized tool invocation. Since `linux-mcp` mediates LLM-triggered access to tools with possible side effects, an error in the admission path could allow an action that should have been denied or deferred. This risk is assigned moderate likelihood ($L = 3$) and major consequence ($C = 4$), giving $R = 12$. Mitigation includes deny-by-default

admission, explicit approval binding, auditable state transitions, and attack-oriented testing.

A second risk is overestimating the system's security guarantees. Kernel placement can strengthen enforcement of specific invariants, but it does not eliminate threats arising from semantic errors in user space, unsafe tool implementations, or incomplete policies. This risk is assigned moderate likelihood ($L = 3$) and very serious consequence ($C = 3$), giving $R = 9$. The mitigation is to define the threat model precisely and to state clearly which protections are provided by kernel enforcement and which remain outside its scope.

A further risk is degraded responsiveness under load. Additional arbitration and approval steps may increase latency or reduce throughput, especially when multiple agents or tools are active concurrently. This risk is assigned moderate likelihood ($L = 3$) and serious consequence ($C = 2$), giving $R = 6$. It should be mitigated through latency analysis, concurrency benchmarking, and optimization of common low-risk paths.

Environmental and resource impact

The direct environmental impact of `linux-mcp` is limited because the project is software-based and does not involve substantial material waste. However, development and evaluation still consume computational resources. Repeated benchmarking, adversarial testing, virtual-machine execution, and debugging of kernel-space behaviour all increase CPU time and electricity consumption. This environmental risk is assigned likely likelihood ($L = 4$) and serious consequence ($C = 2$), giving $R = 8$. The appropriate mitigation is careful experiment planning, automation of result collection, and avoidance of redundant reruns.

A practical project-management risk is loss of development time and compute resources due to unstable experiments, failed integration, or repeated debugging. This risk is assigned likely likelihood ($L = 4$) and serious consequence ($C = 2$), giving $R = 8$. Mitigation includes reproducible scripts, checkpointing of experiment outputs, staged deployment, and careful configuration management.

Another resource-related risk is dependence on external tooling or infrastructure. The project may rely on LLM APIs, network configuration, proxies, virtual machines, or other software dependencies during development and evaluation. This risk is assigned moderate likelihood ($L = 3$) and serious consequence ($C = 2$), giving $R = 6$. This risk can be reduced by separating the core evaluation from optional cloud dependencies and by supporting local or mocked execution paths where possible.

Table 12: Summary of identified risks and mitigation strategies for `linux-mcp`.

Risk	*L*	*C*	*R*	**Rating**	**Mitigation**
Kernel-level instability	3	4	12	Significant	Narrow kernel scope, strict interfaces, staged testing, stress testing
Weak justification over user-space baselines	4	3	12	Significant	Strong baselines, adversarial evaluation, quantitative comparison
Cross-component integration failure	3	2	6	Moderate	Explicit protocols, schema synchronization, end-to-end validation
Unsafe or unauthorized tool invocation	3	4	12	Significant	Deny-by-default admission, approval binding, auditability, misuse testing
Overestimated security guarantees	3	3	9	Significant	Precise threat model, careful claims, clear scope of guarantees
Added latency or reduced throughput	3	2	6	Moderate	Latency analysis, concurrency evaluation, fast-path optimization
Additional compute and energy use	4	2	8	Significant	Efficient experiment planning, automation, fewer redundant reruns
Loss of development time and compute resources	4	2	8	Significant	Reproducible workflows, checkpointing, staged deployment
External dependency or infrastructure failure	3	2	6	Moderate	Local or mocked backends, dependency isolation, reduced cloud reliance